Introduction to Inexpensive Alternatives for Improved Deep Learning-based Side-channel Analysis

Azade Rezaeezade – Lizzy Grootjen

October 11, 2026

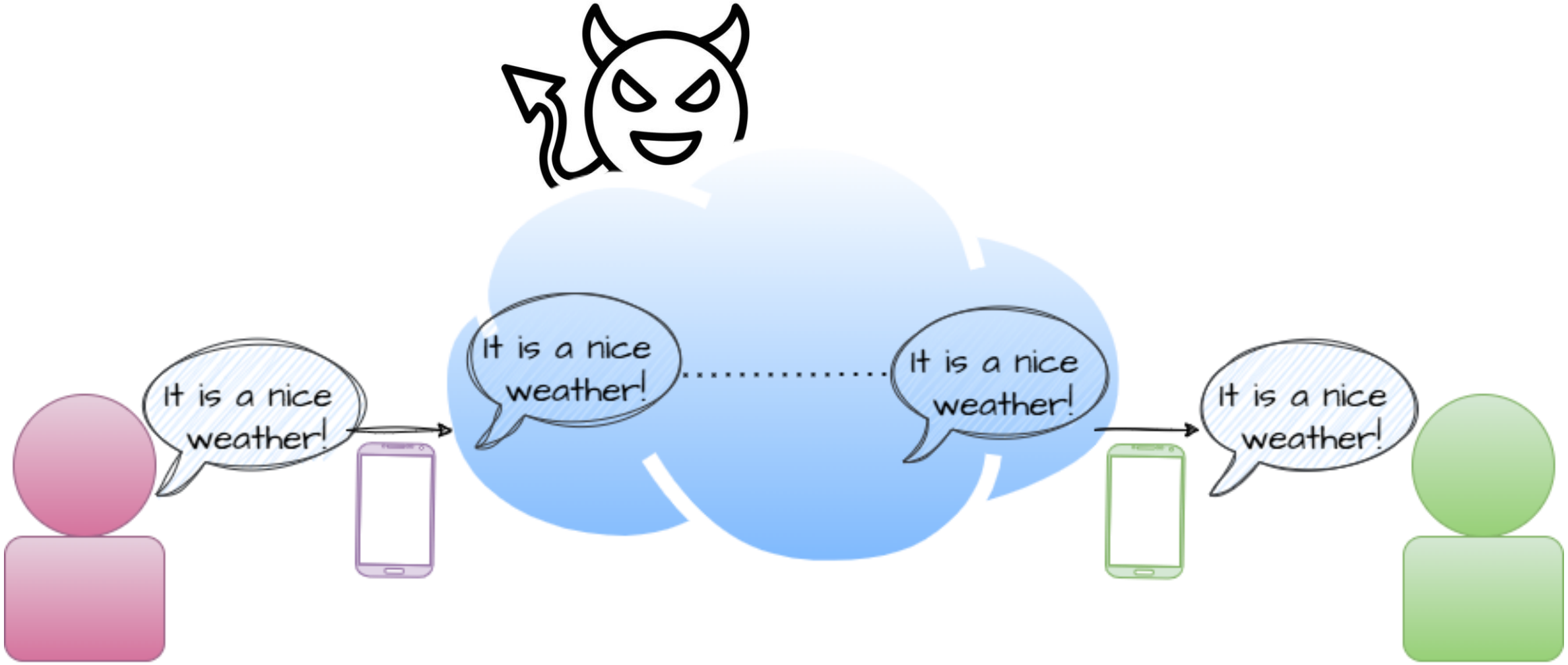
Agenda

- First session 13:30-15:00
  - Introduction to DL-SCA
  - Ensemble learning
  - Multitask learning
- Coffee Break 15:00-15:30
- Second session 15:30-17:00
  - Regularization techniques
  - Customized loss and activation
  - Wrap-up and Q&A

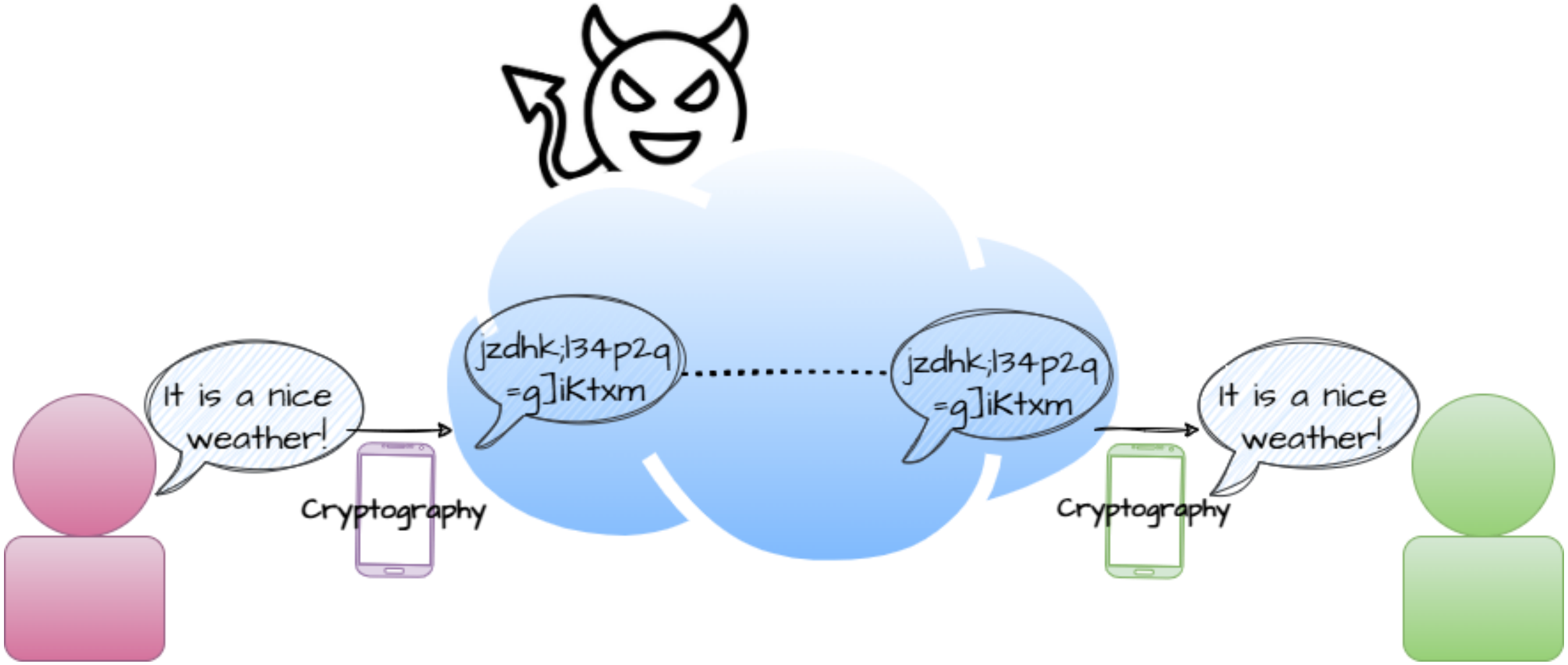
Deep Learning-Based Side-Channel Analysis



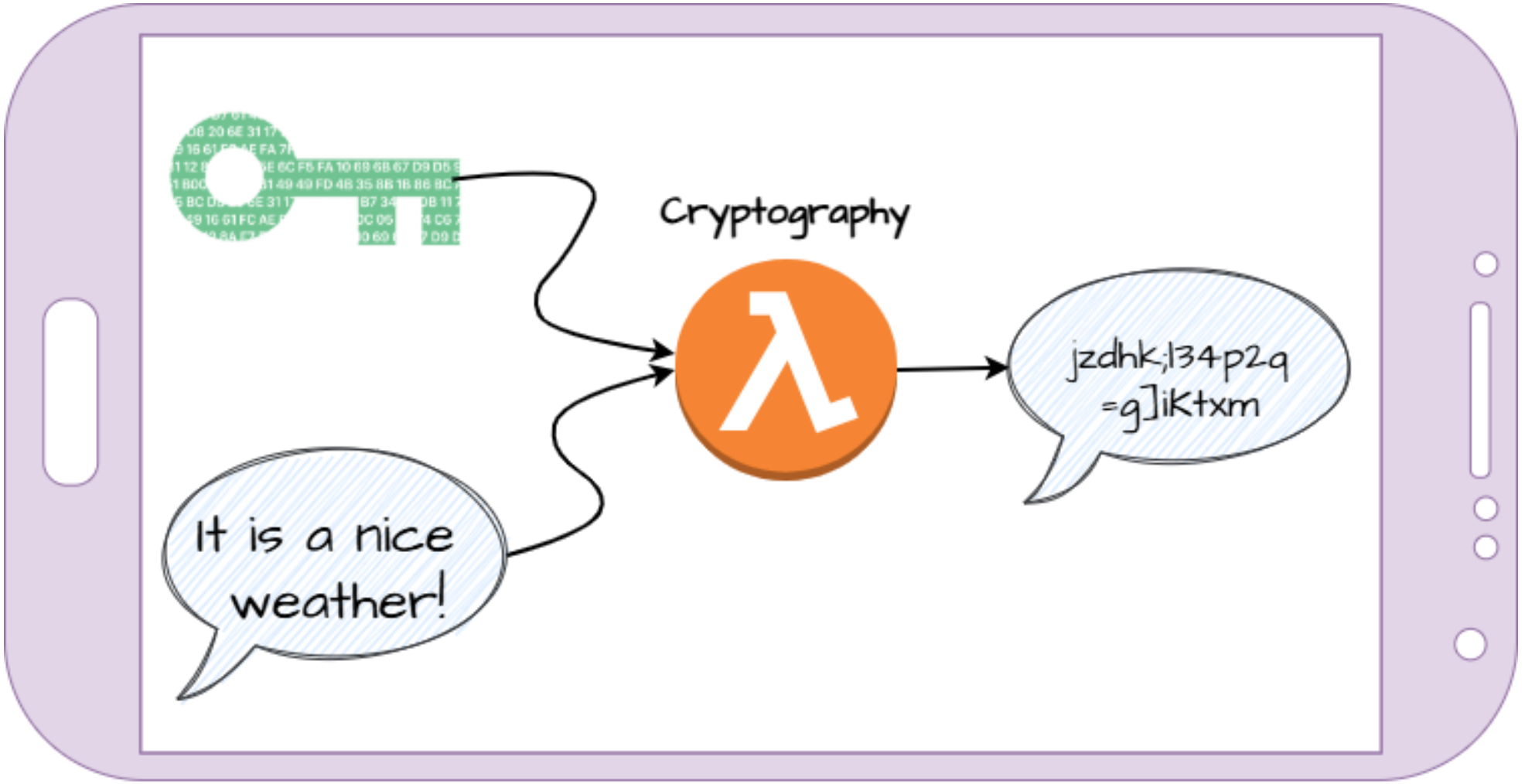
Cryptography



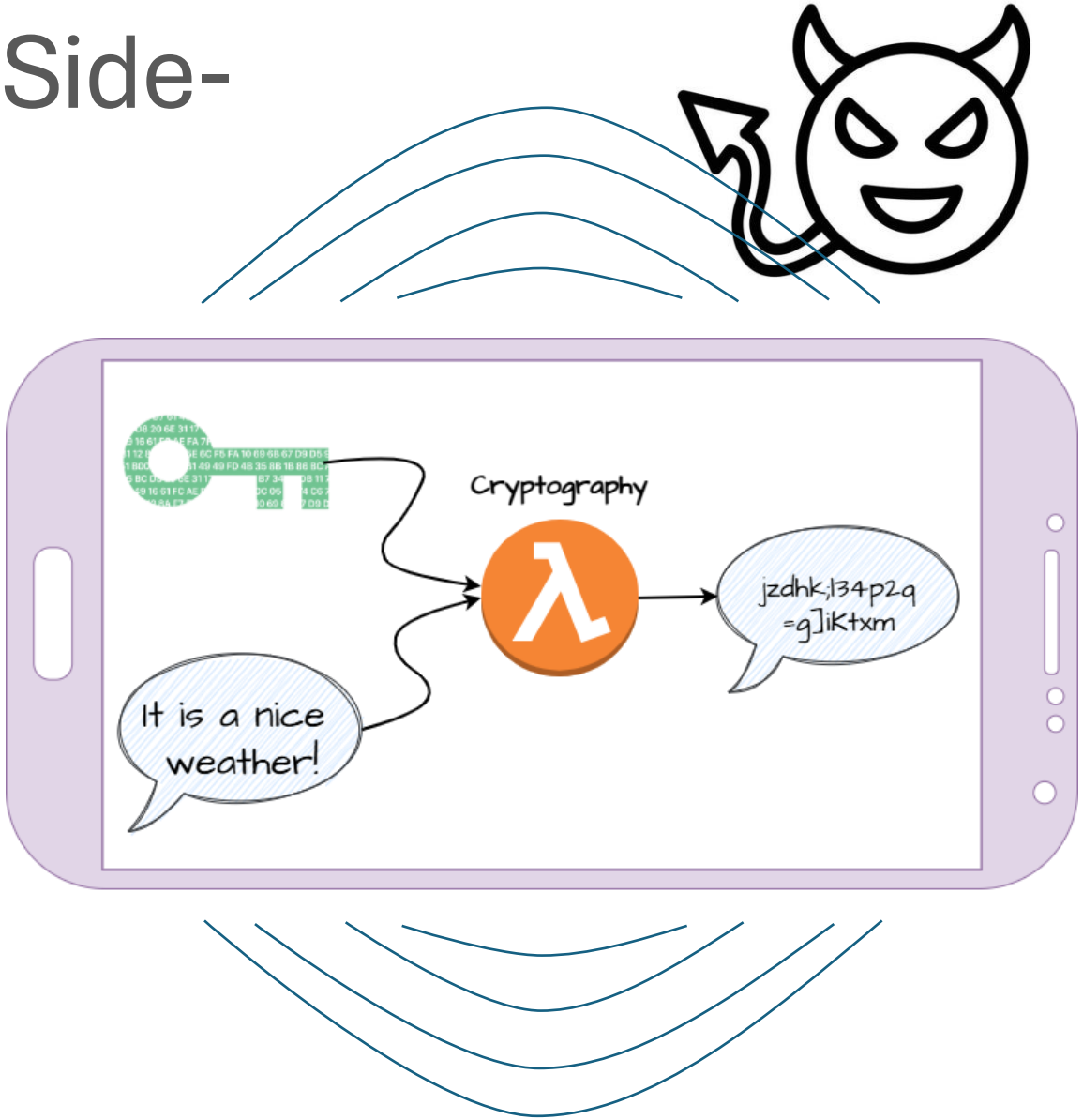
Cryptography



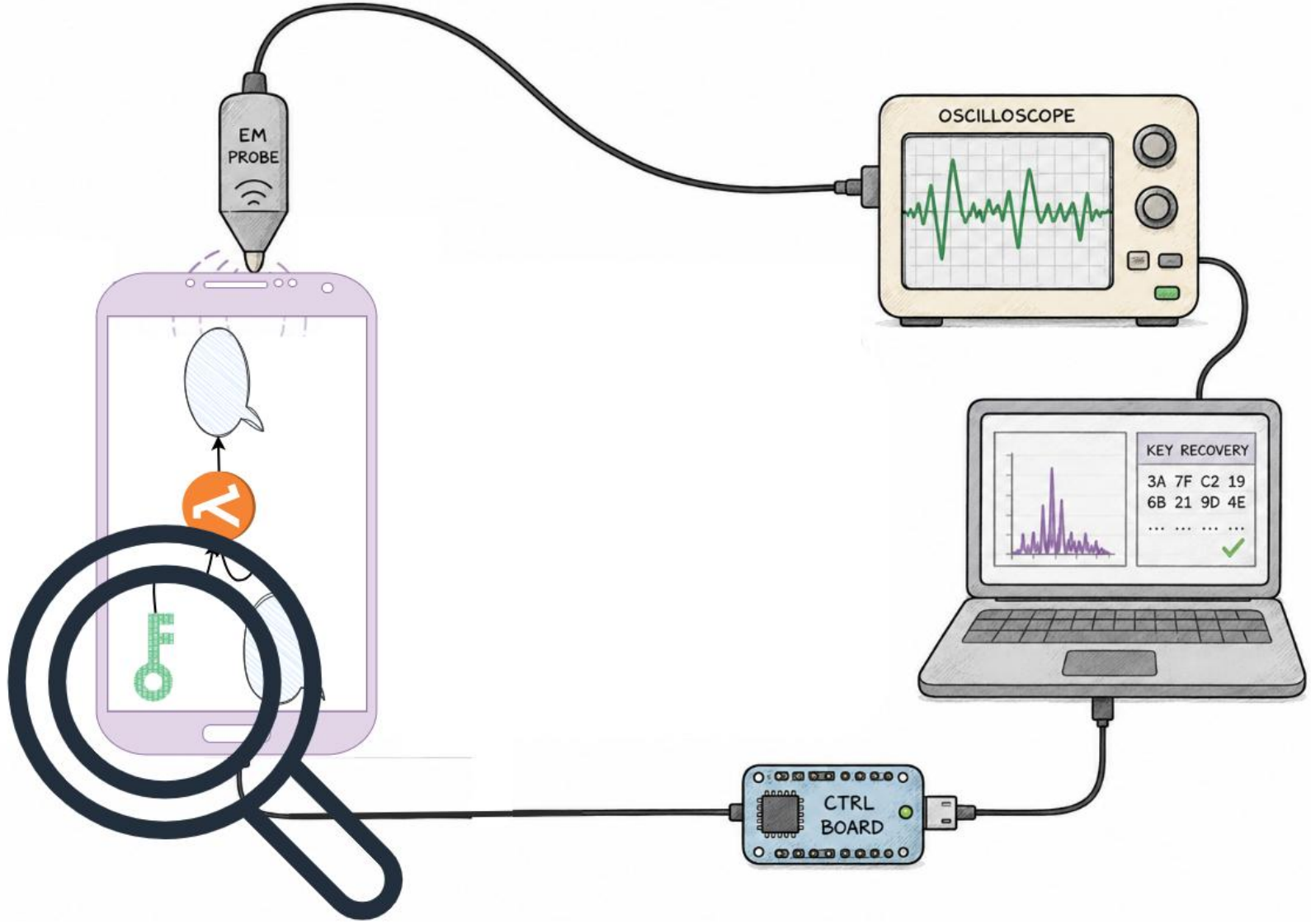
Cryptography



Leakage & Side-Channels



Measurement



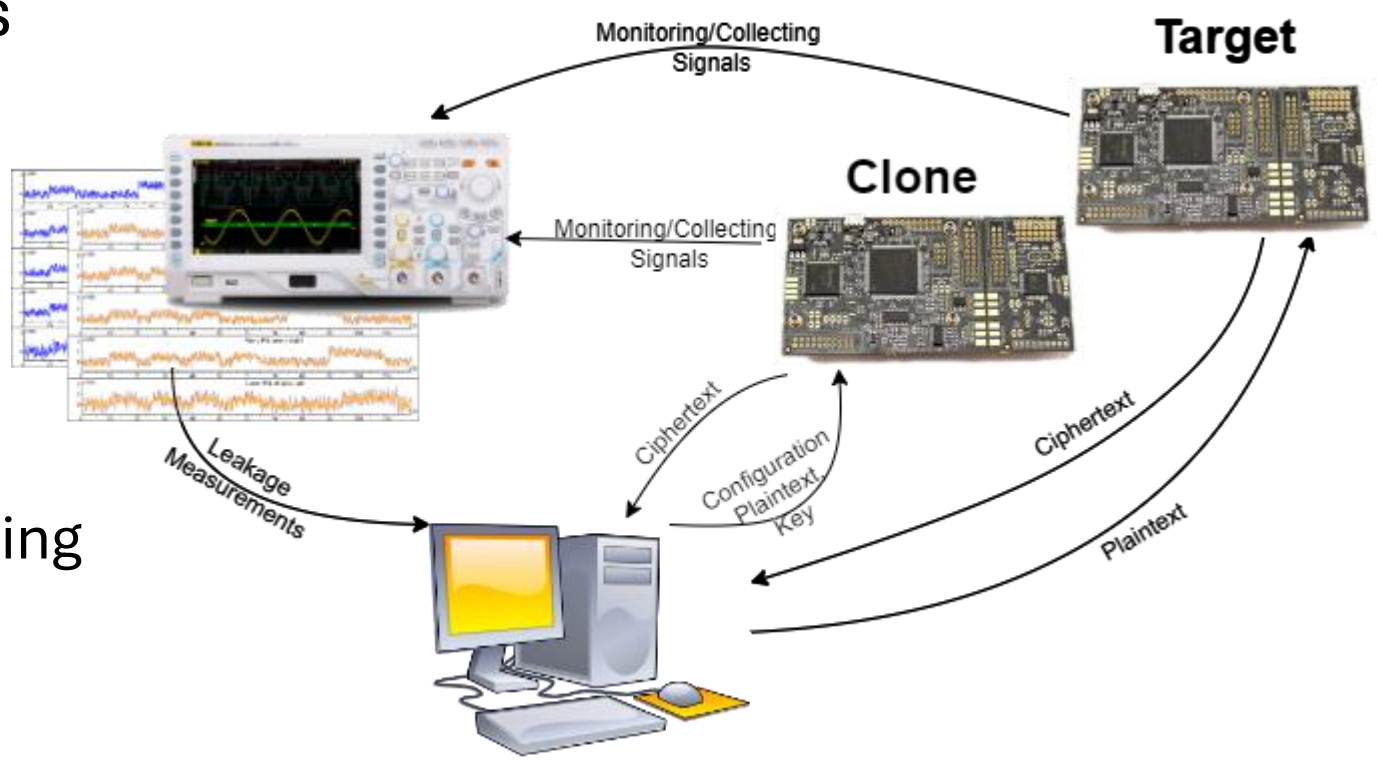
Side-Channel Attack

Side-channel Analysis

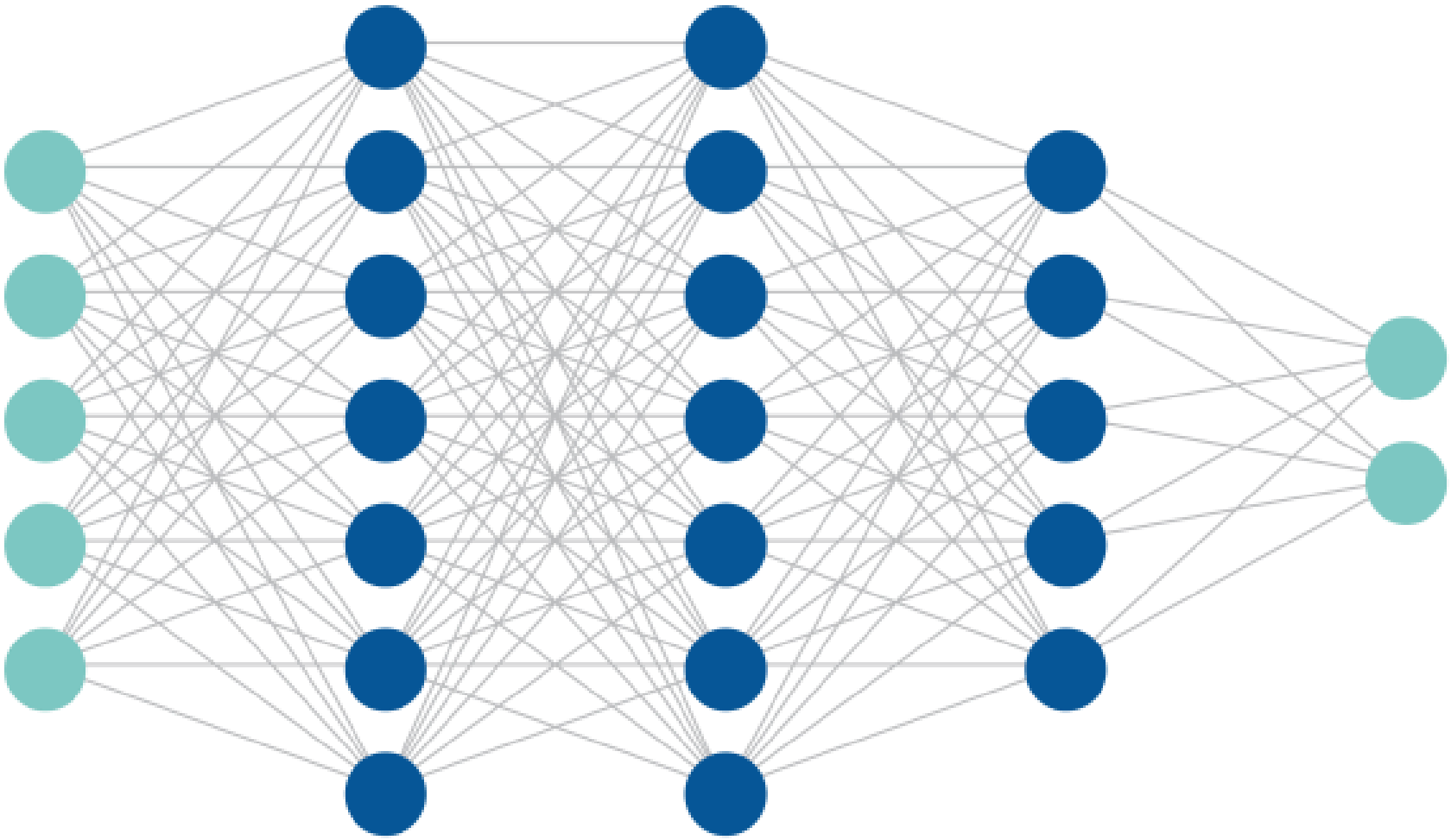
- Non-profiling
- Profiling

Profiling SCA phases:

- Profiling
- Attack
- Machine/Deep Learning

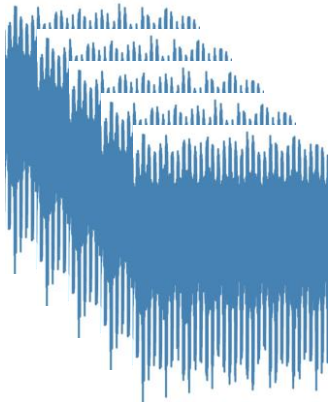


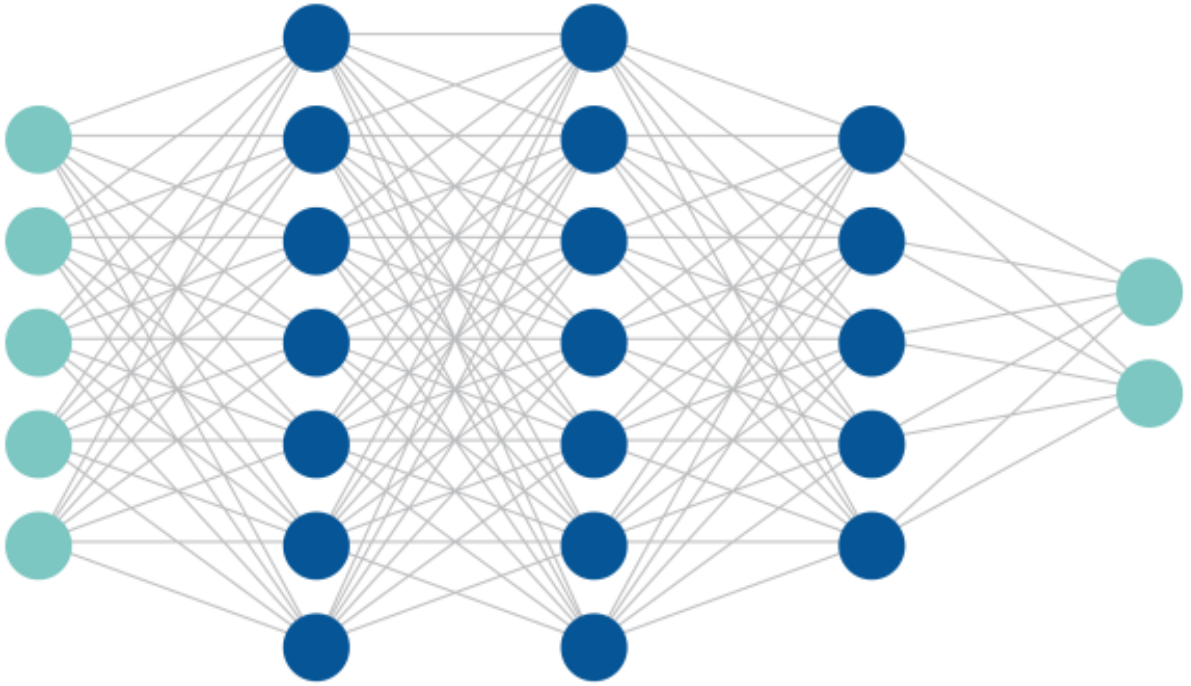
Deep Learning-based Side-channel Analysis



Deep Learning-based Side-channel Analysis

Traces





Classes

Which are defined by:

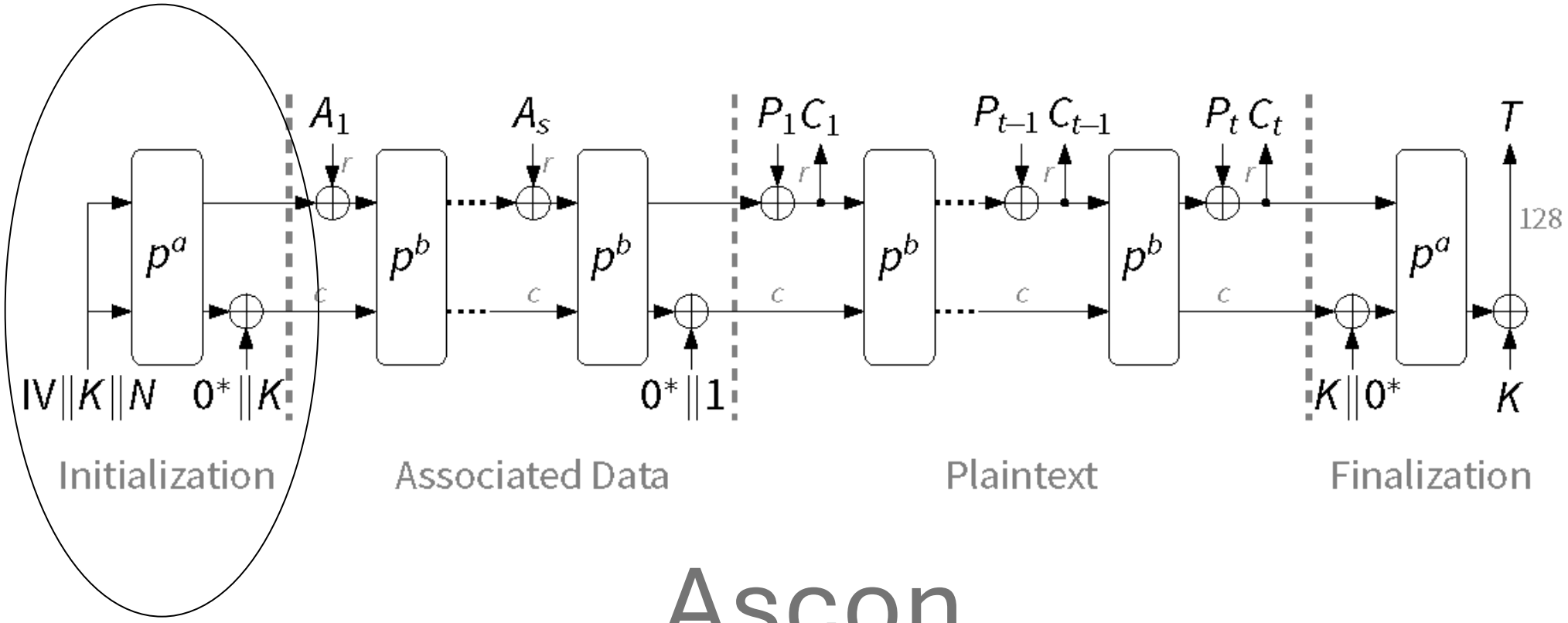
- An intermediate value
- A leakage model

How good the model is?

DL-SCA metrics:

- Guessing Entropy
- Required number of attack traces

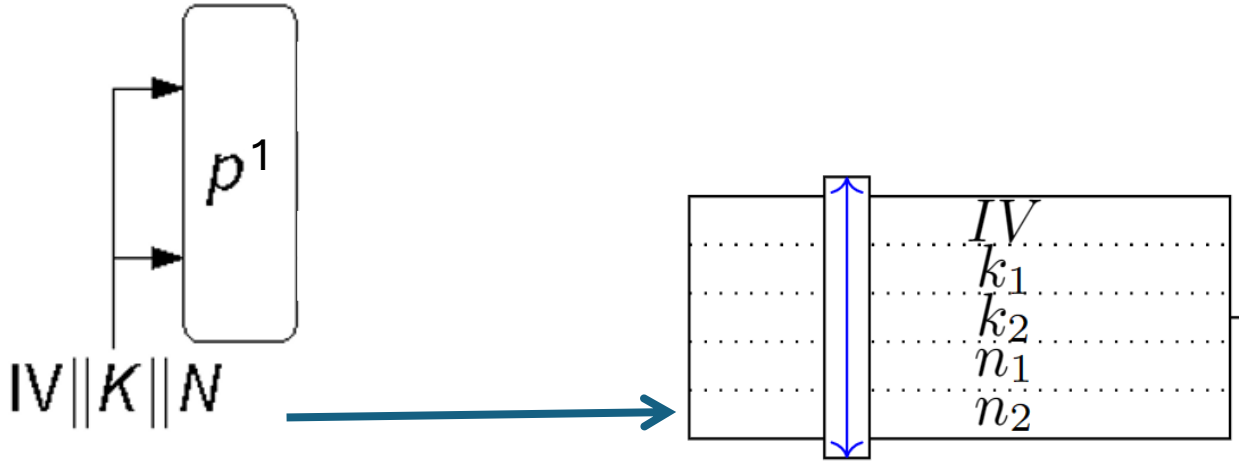
Intermediate Value



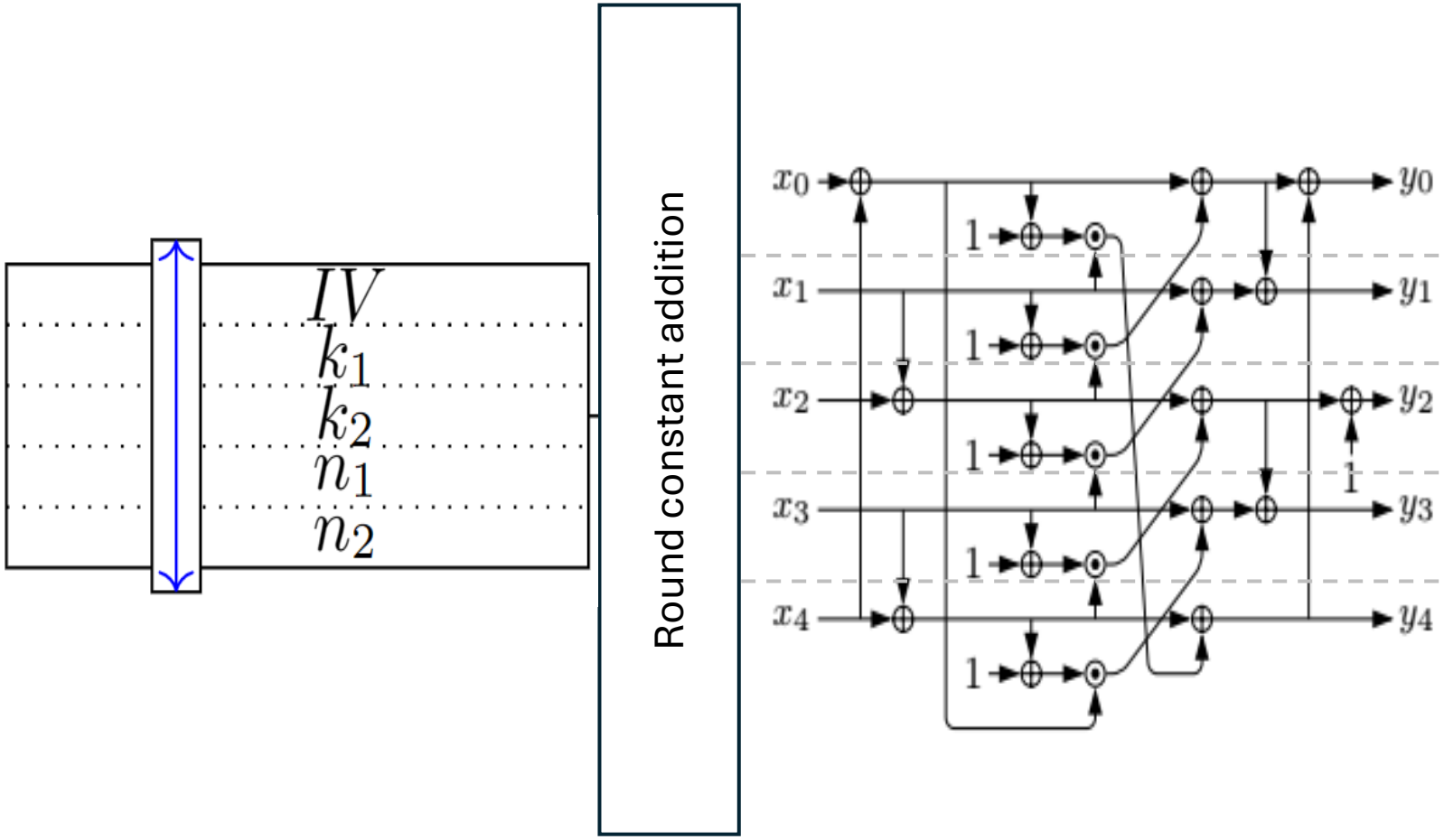
Ascon

Algorithm Input

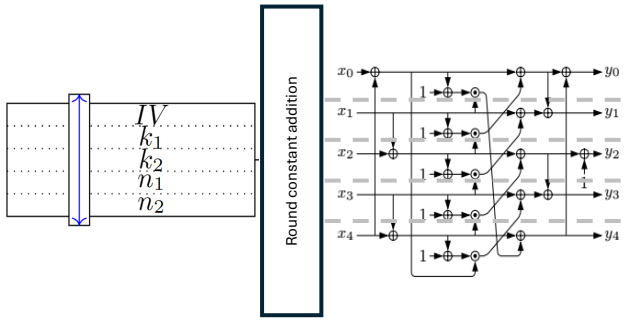
- 12 rounds of the same permutation
- 320-bit initial state



Attack Target: The S-box Output



Attack Target: The S-box Output



$$y_0 = x_0 + x_1 + x_2 + x_3 + x_1x_2 + x_0x_1 + x_1x_4$$

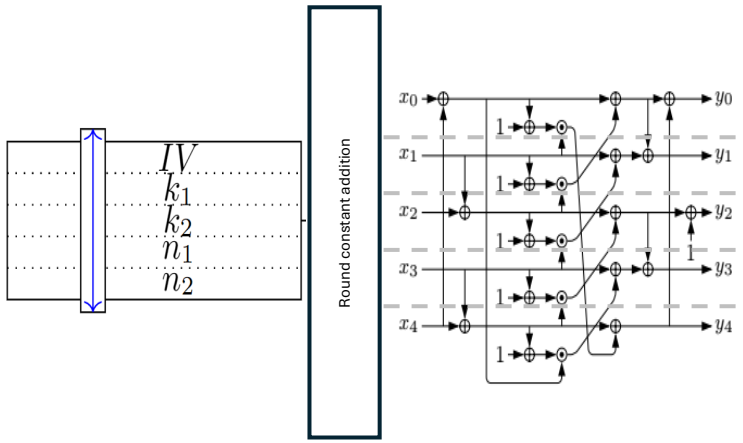
$$y_1 = x_0 + x_1 + x_2 + x_3 + x_4 + x_1x_2 + x_1x_3 + x_2x_4$$

$$y_2 = x_1 + x_2 + x_4 + x_3x_4 + 1$$

$$y_3 = x_0 + x_1 + x_2 + x_3 + x_4 + x_0x_3 + x_0x_4$$

$$y_4 = x_1 + x_3 + x_4 + x_0x_1 + x_1x_4$$

Intermediate Value



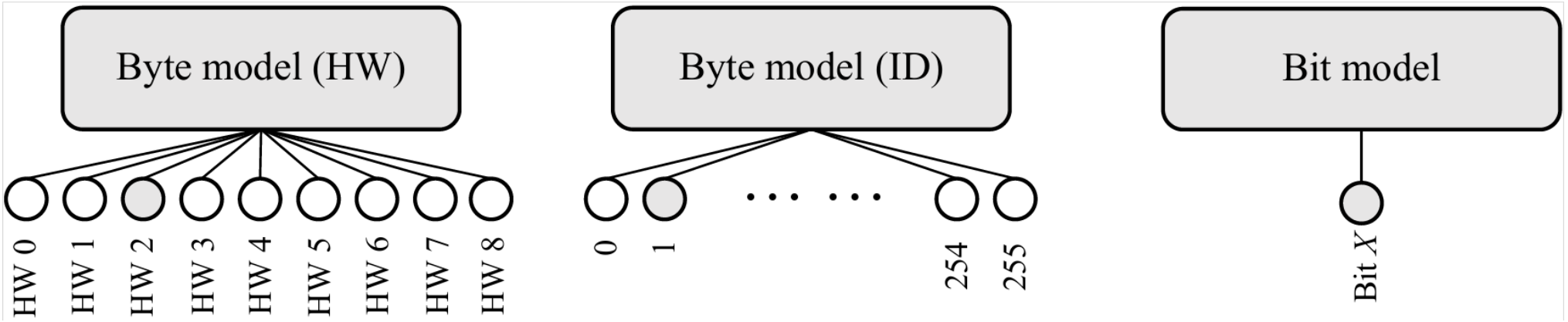
$$y_4 = x_1 + x_3 + x_4 + x_0x_1 + x_1x_4$$
$$= x_1(1 + x_0 + x_4) + x_3 + x_4$$

$$Y_4 = [key0\&((2^{64} - 1) \oplus IV \oplus Nonce1)] \oplus Nonce0 \oplus Nonce1$$

$$Y_{4,i} = [\mathrm{key0}_i\&(255 \oplus \mathrm{IV}_i \oplus \mathrm{nonces1}_i)] \oplus \mathrm{nonces0}_i \oplus \mathrm{nonces1}_i \qquad for\ i\ \in \{0, \dots, 7\}$$

Leakage Model

- The pattern that describes the intermediate value leakage



Attack Metrics

- Guessing entropy reports the actual key rank among all the possible keys after the attack

$$S_k(n) = \sum_{i=1}^{n} \log(p_i[f(k, \text{nonce}_i)] + \varepsilon)$$

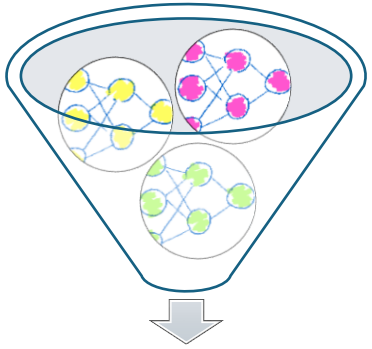
$$\text{GE}(n) = \frac{1}{R} \sum_{r=1}^{R} S_{r,n}(k^\star)$$

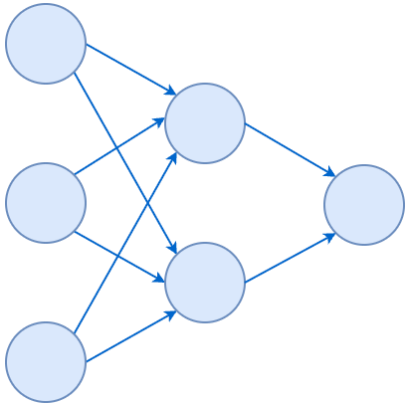
- Required number of attack traces reports the number of traces we need for attack to always place actual key on rank one


ensemble

Pool of Models

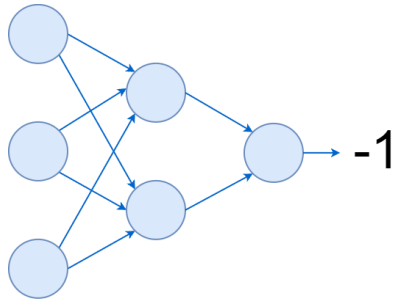
- Hyperparameter tuning
- Pool of models
- A single best model

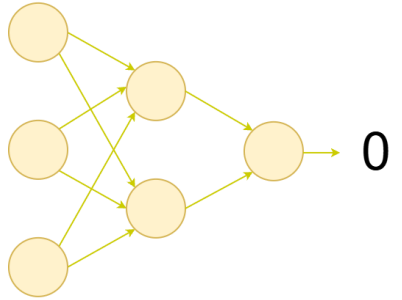


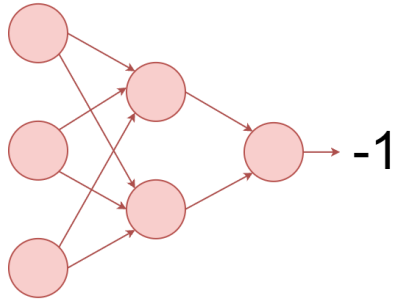


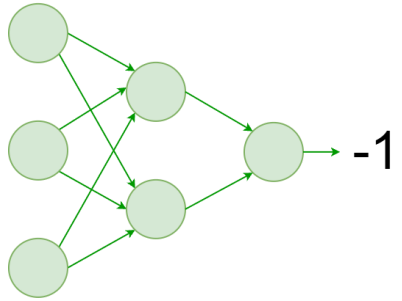
Ensemble Learning

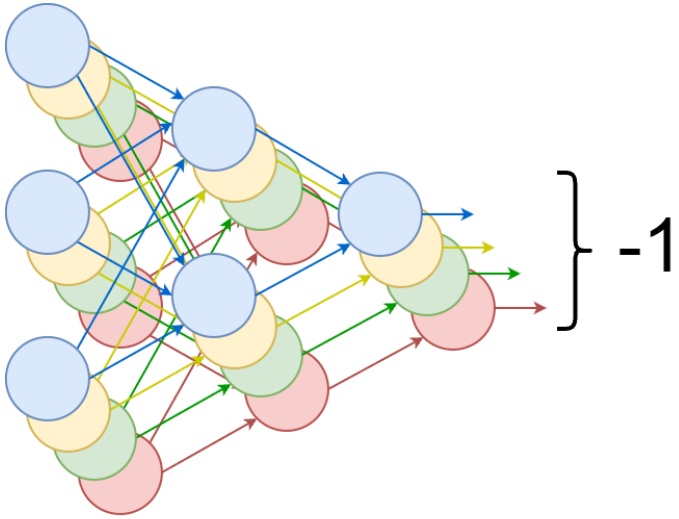
- Ensemble learning
- Sub-optimal models
- No extra resource is needed
  - Example for 2 classes classification
  - Majority voting between the neural networks predictions





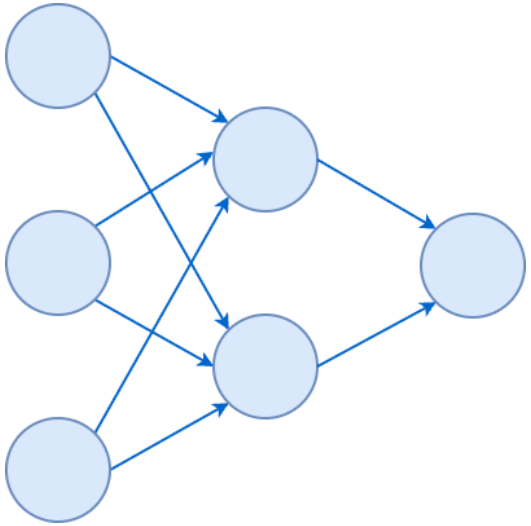






Ensemble Learning for Side-Channel

- Guessing Entropy (GE) definition



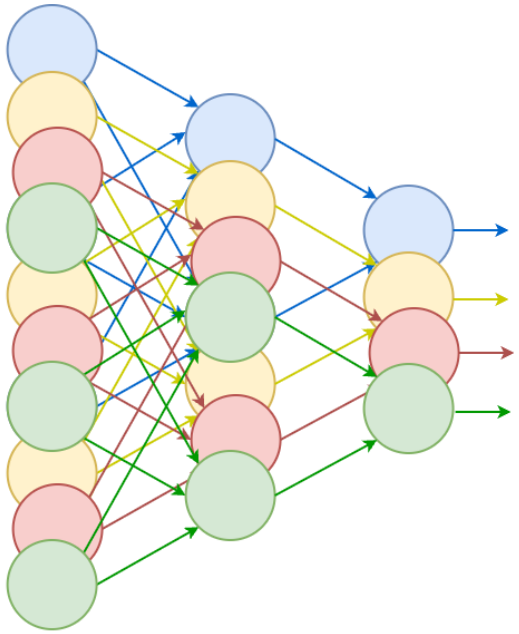
$$\sum \log \downarrow \begin{matrix} \begin{bmatrix} p_{0,0} & p_{0,1} & \cdots & p_{0,c-1} \\ \vdots & & \ddots & \vdots \\ p_{n,0} & p_{n,1} & \cdots & p_{n,c-1} \end{bmatrix} \\ \hline P_0 \quad P_1 \quad \cdots \quad P_{c-1} \end{matrix}$$

Sort

$$P_k \quad P_{c-2} \quad \cdots \quad P_1$$

$GE = the\ average\ position\ of\ the$ actual key

Ensemble Learning for SCA



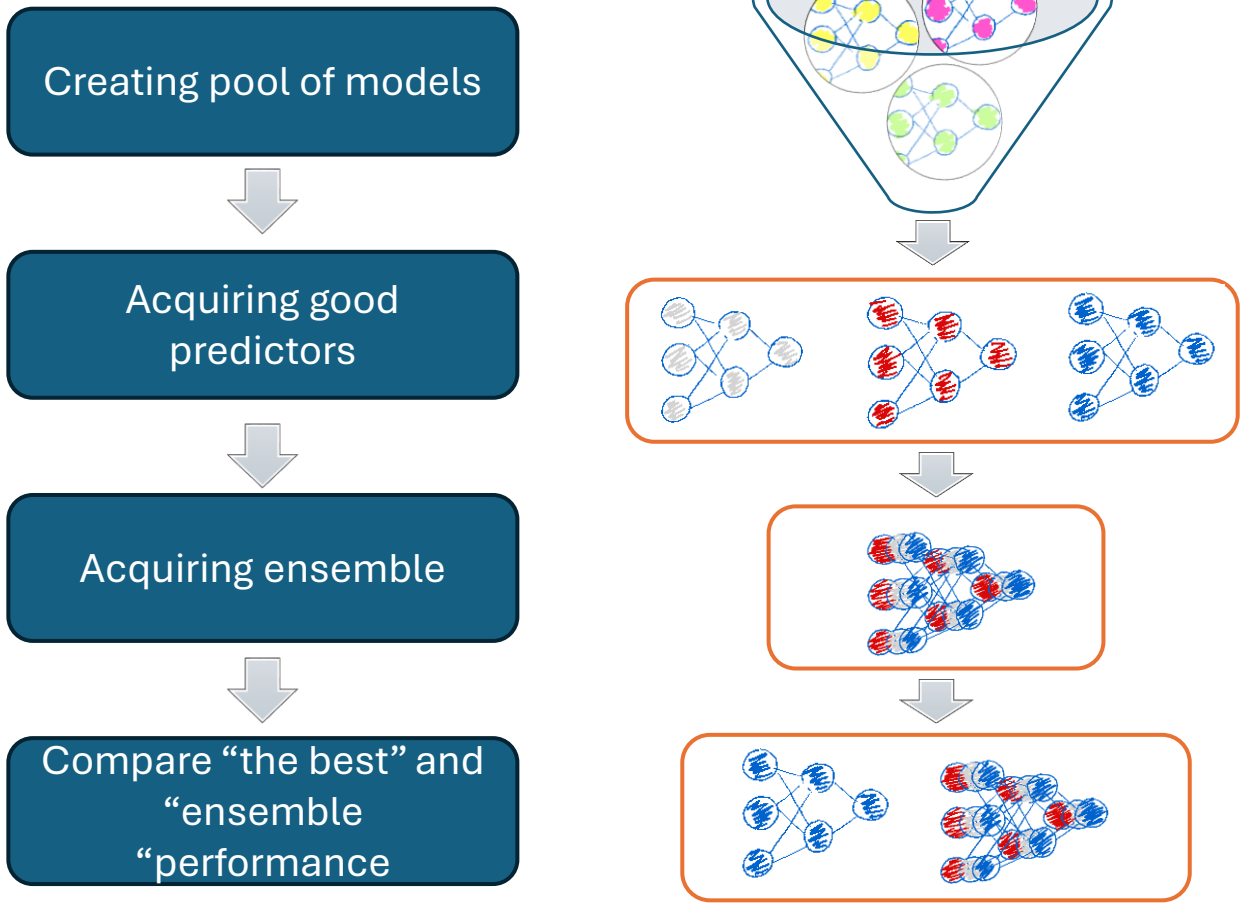
$$\begin{bmatrix} p_{0,0} & p_{0,1} & \cdots & p_{0,c-1} \\ \vdots & & \ddots & \vdots \\ p_{n,0} & p_{n,1} & \cdots & p_{n,c-1} \end{bmatrix} \begin{bmatrix} q_{0,0} & q_{0,1} & \cdots & q_{0,c-1} \\ \vdots & & \ddots & \vdots \\ q_{n,0} & q_{n,1} & \cdots & q_{n,c-1} \end{bmatrix} \begin{bmatrix} r_{0,0} & r_{0,1} & \cdots & r_{0,c-1} \\ \vdots & & \ddots & \vdots \\ r_{n,0} & r_{n,1} & \cdots & r_{n,c-1} \end{bmatrix}$$

$$\sum \log \rightarrow \frac{\begin{bmatrix} s_{0,0} & s_{0,1} & \cdots & s_{0,c-1} \\ \vdots & & \ddots & \vdots \\ s_{n,0} & s_{n,1} & \cdots & s_{n,c-1} \end{bmatrix}}{\sum \log \downarrow \quad S_0 \quad S_1 \quad \cdots \quad S_{c-1}}$$

$$\xrightarrow{\text{Sort}} \quad S_k \quad S_{c-2} \quad \cdots \quad S_1$$

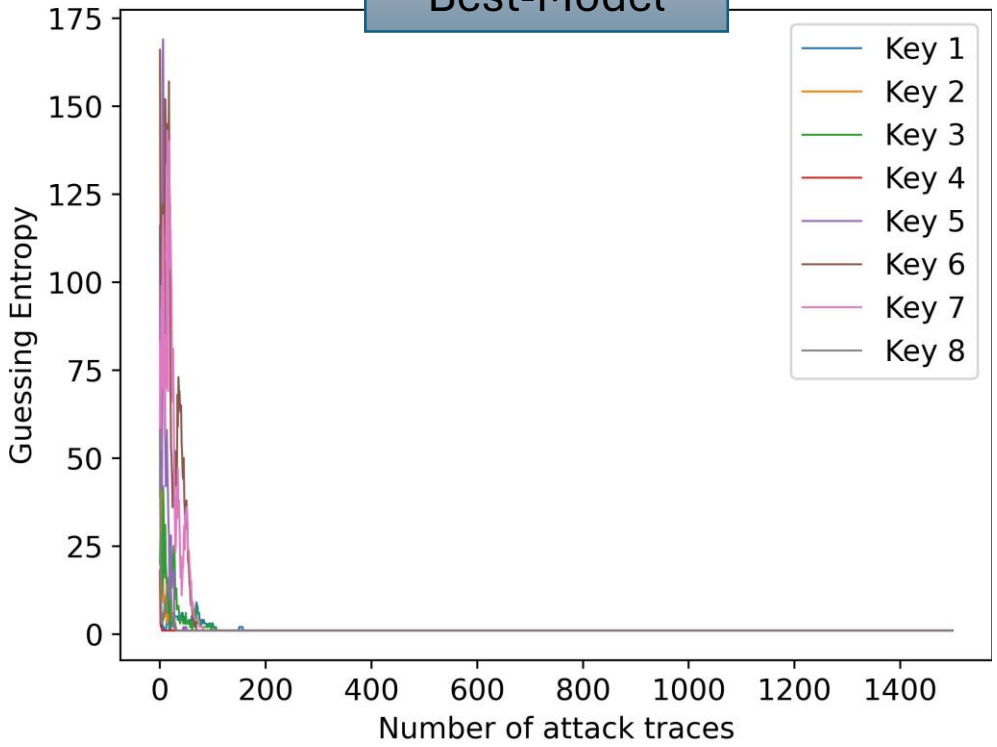
Methodology

- Pool of 60 models
- Using random search
- Select 5 best
- Performance metrics
  - Guessing entropy
  - Required attack traces

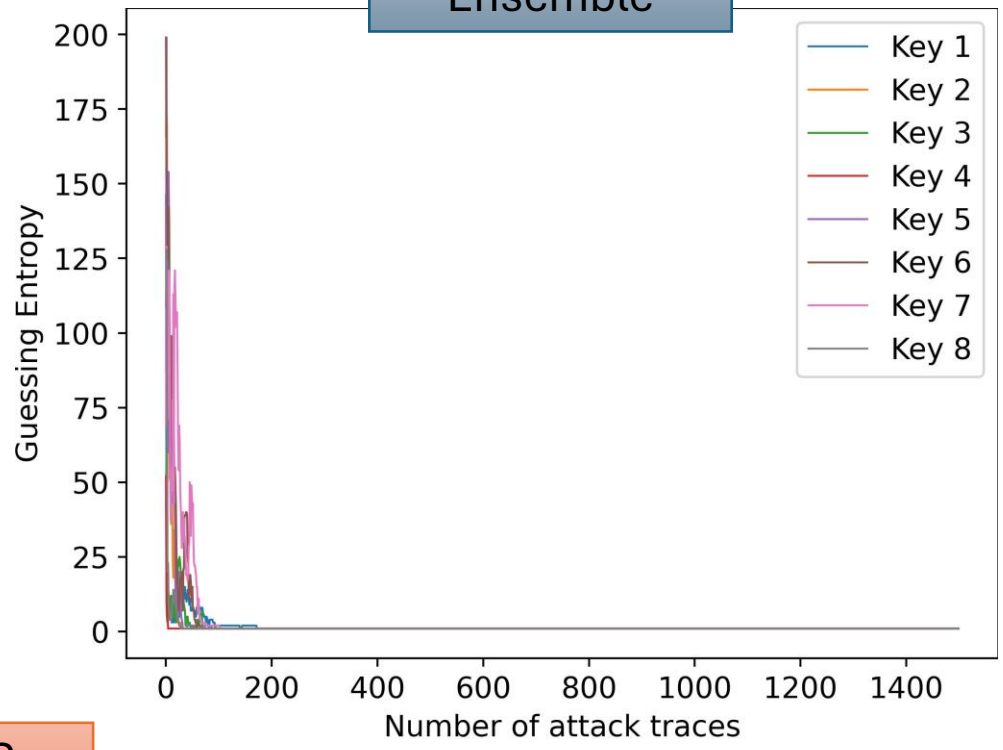


Guessing Entropy: Unprotected

Best-Model



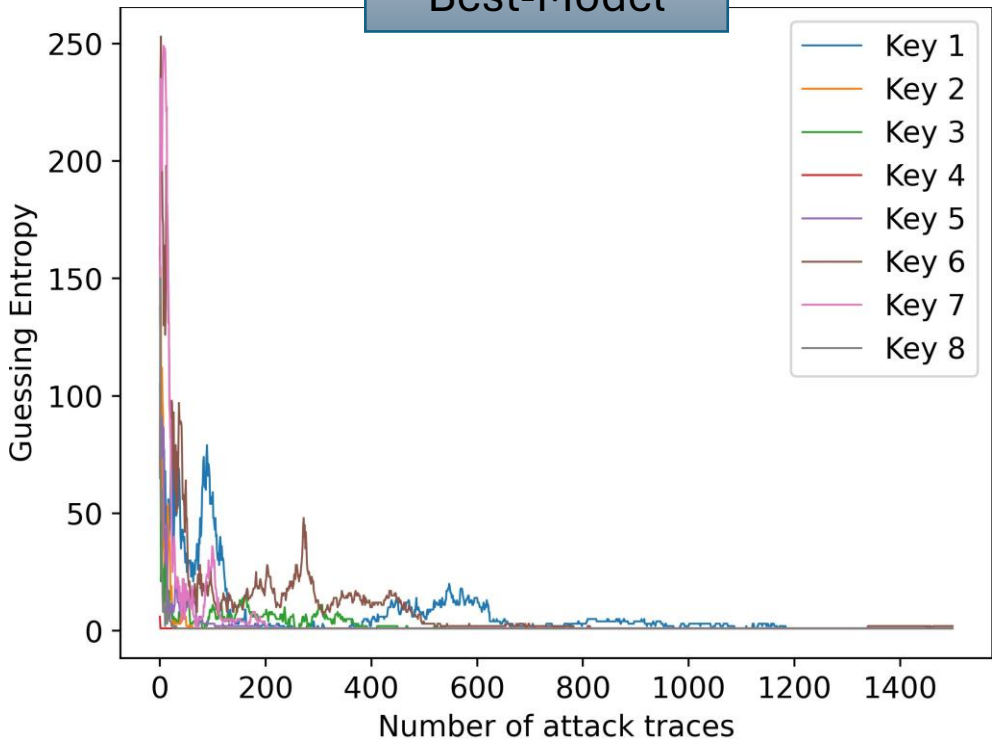
Ensemble



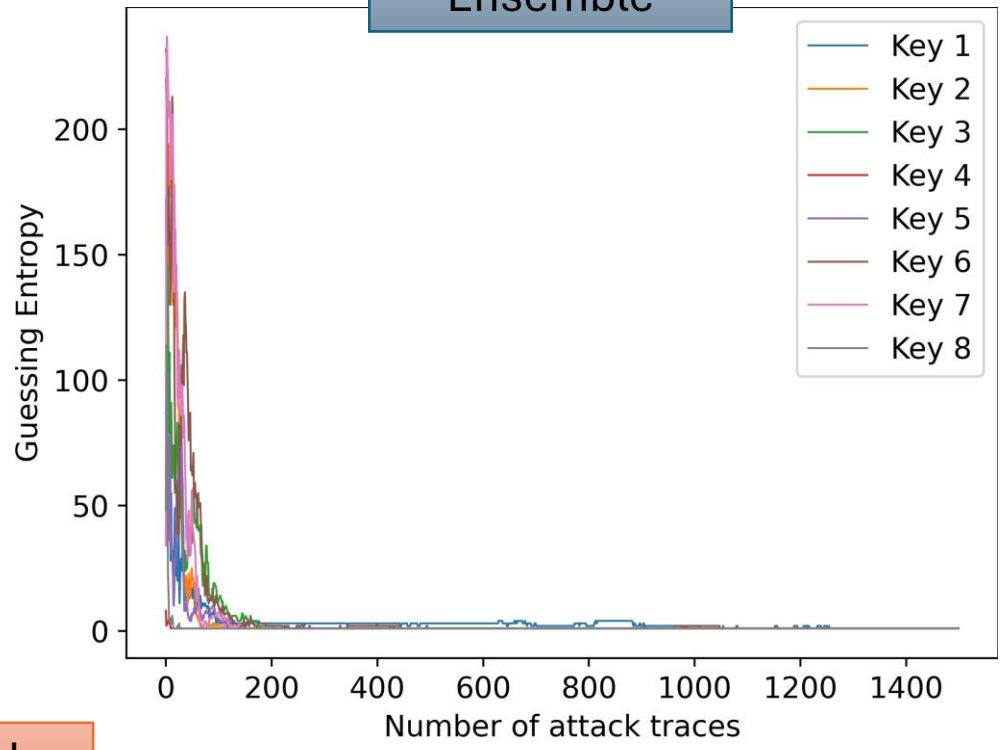
MLP

Guessing Entropy: Unprotected

Best-Model

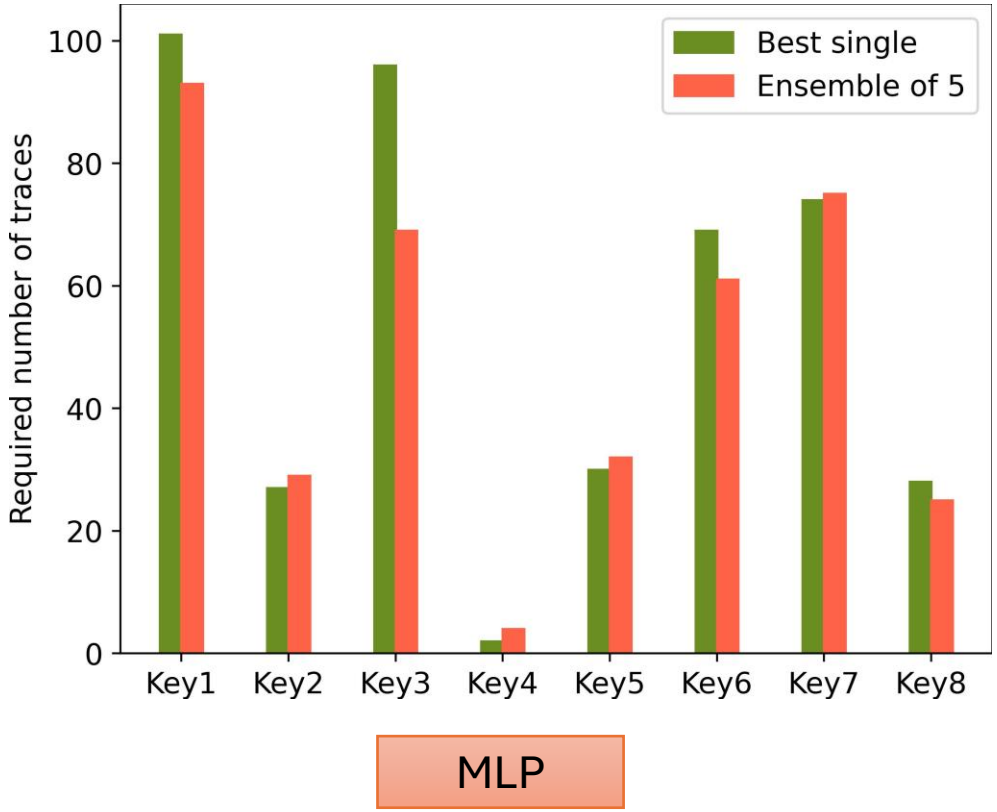


Ensemble

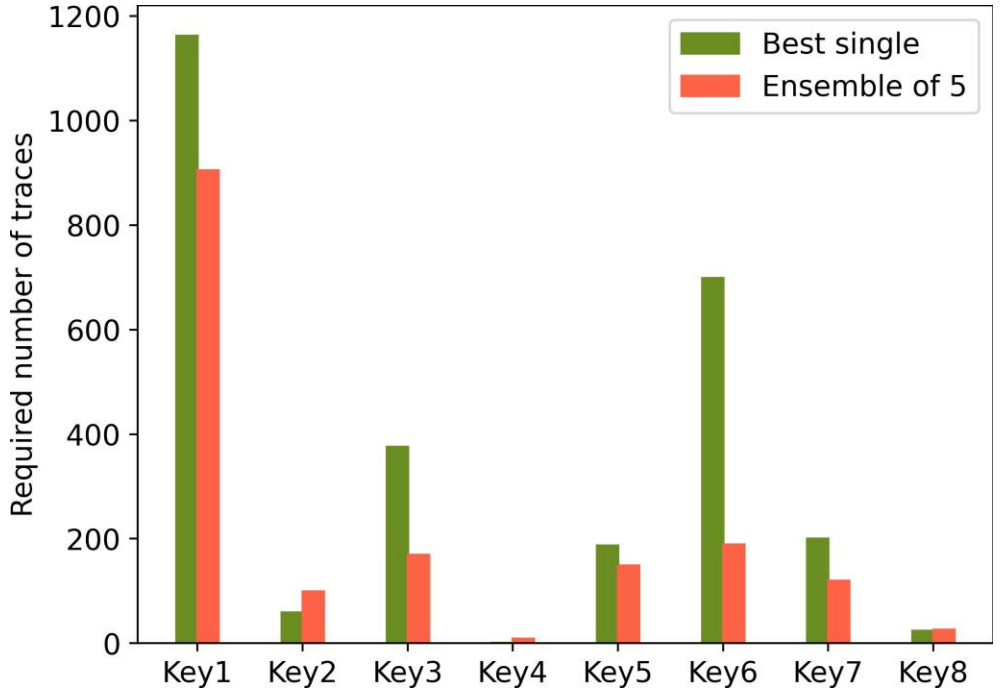


CNN

Required Traces: Unprotected



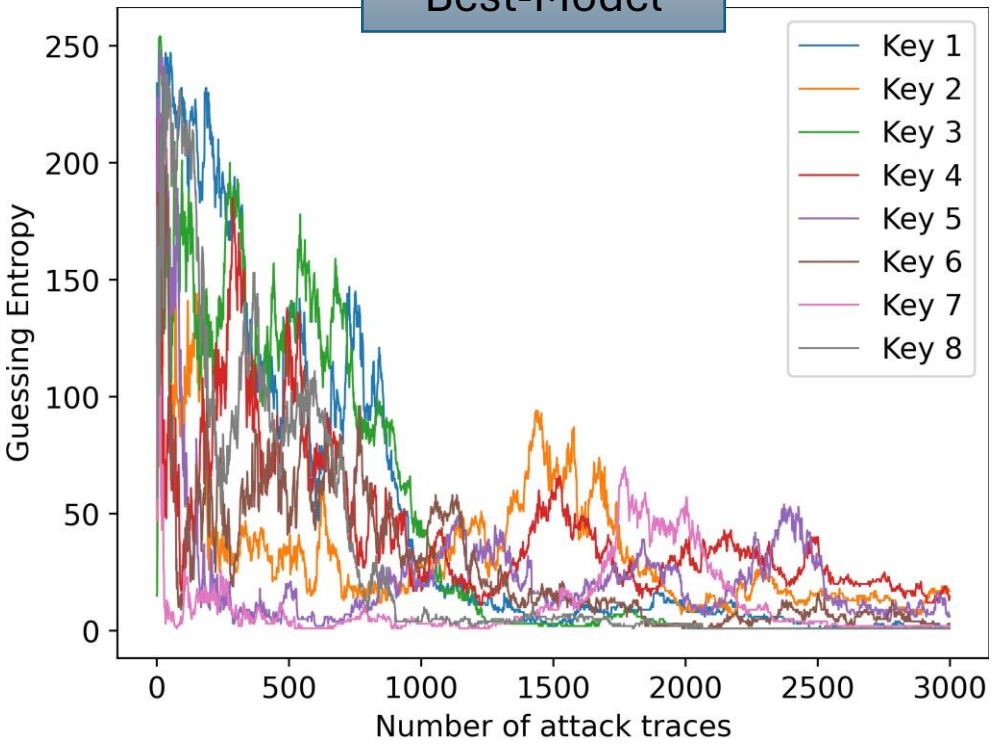
MLP



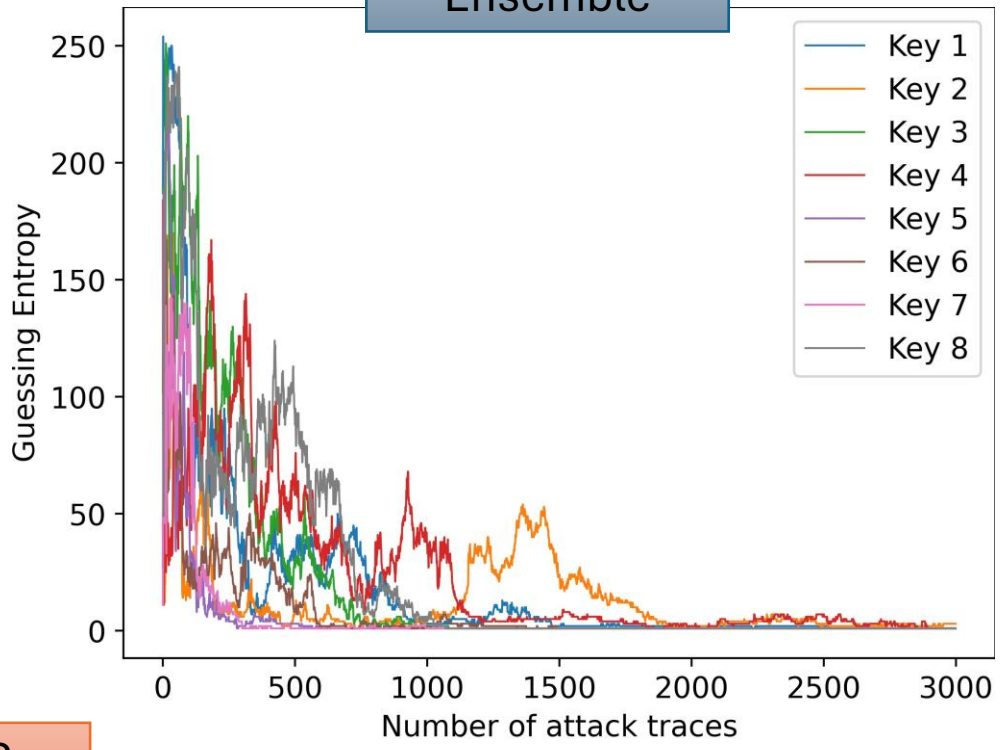
CNN

Guessing Entropy: Protected

Best-Model

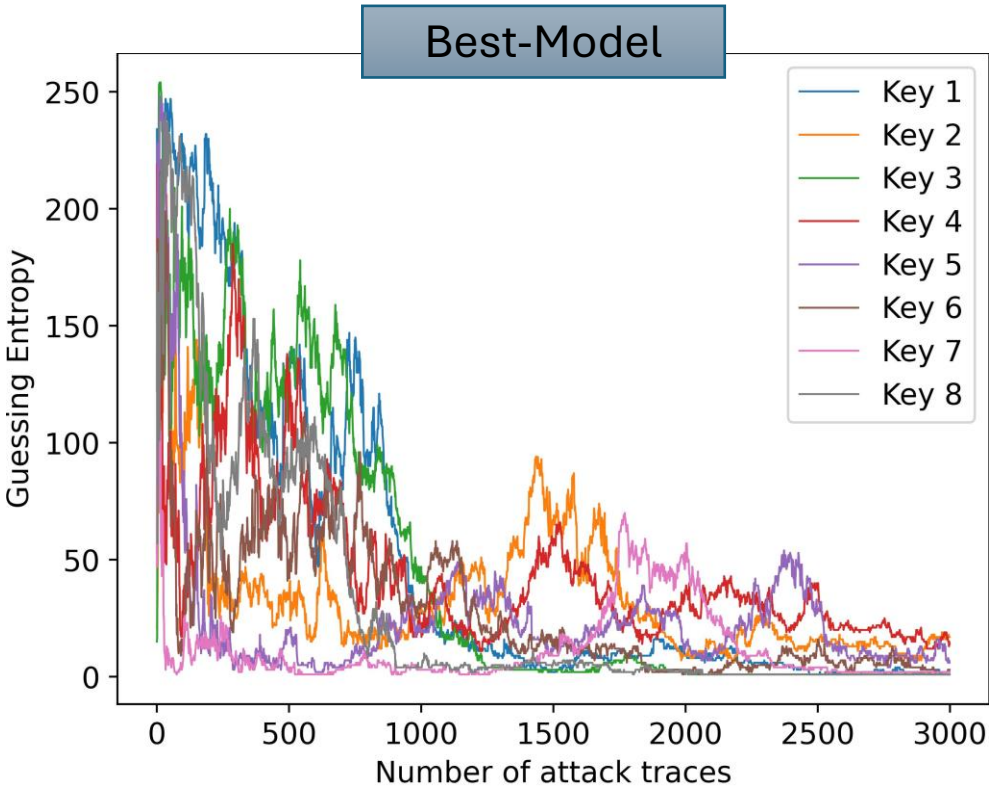


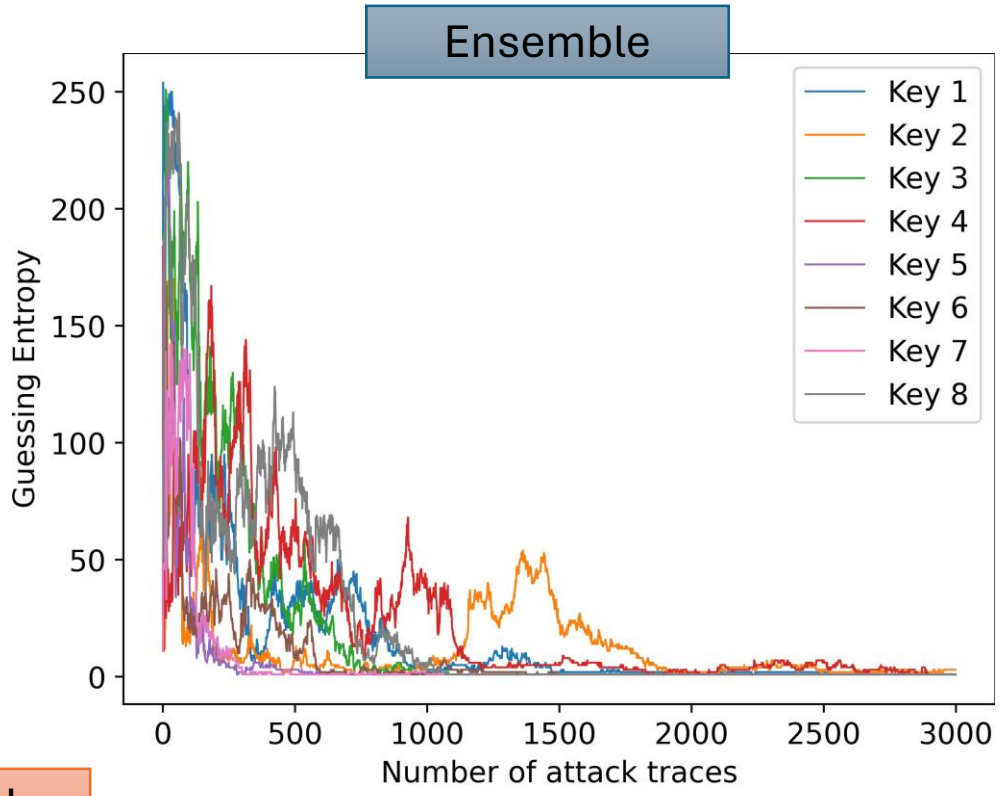
Ensemble



MLP

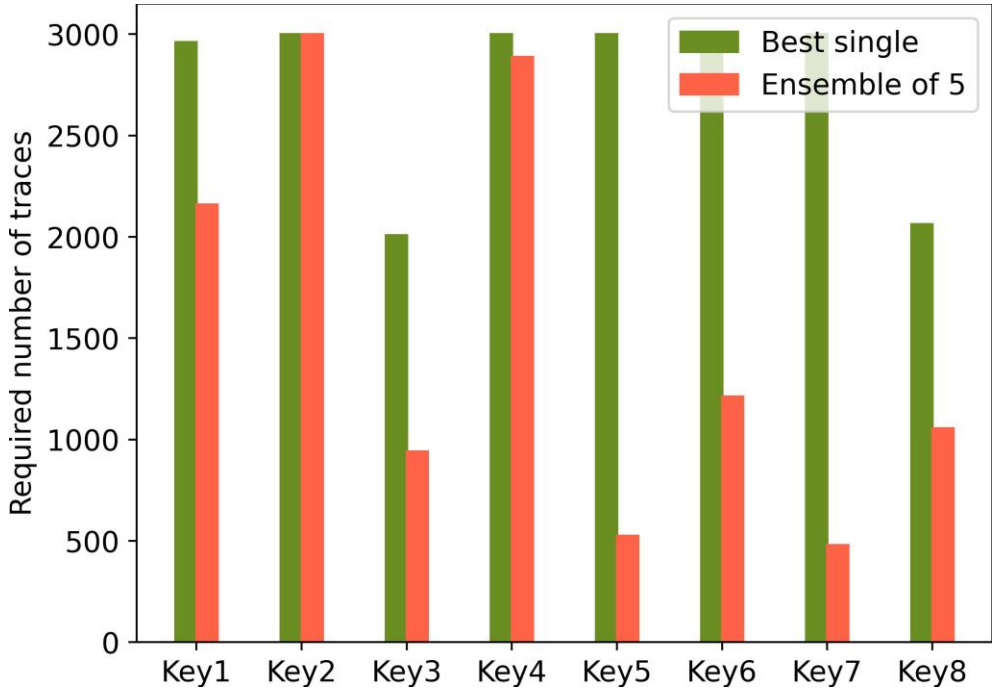
Guessing Entropy: Protected



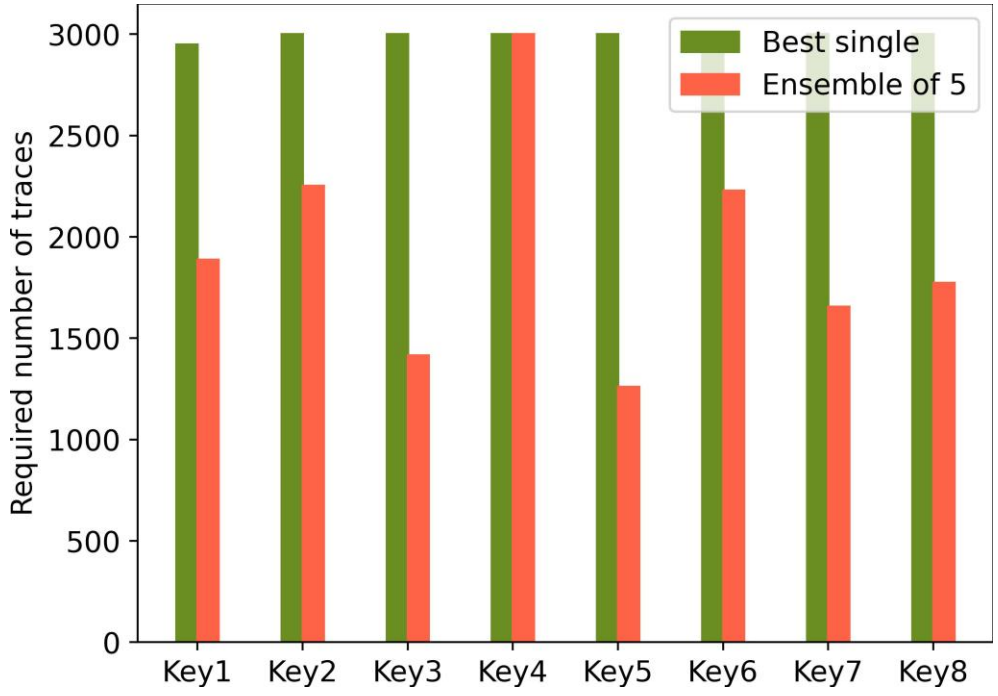


CNN

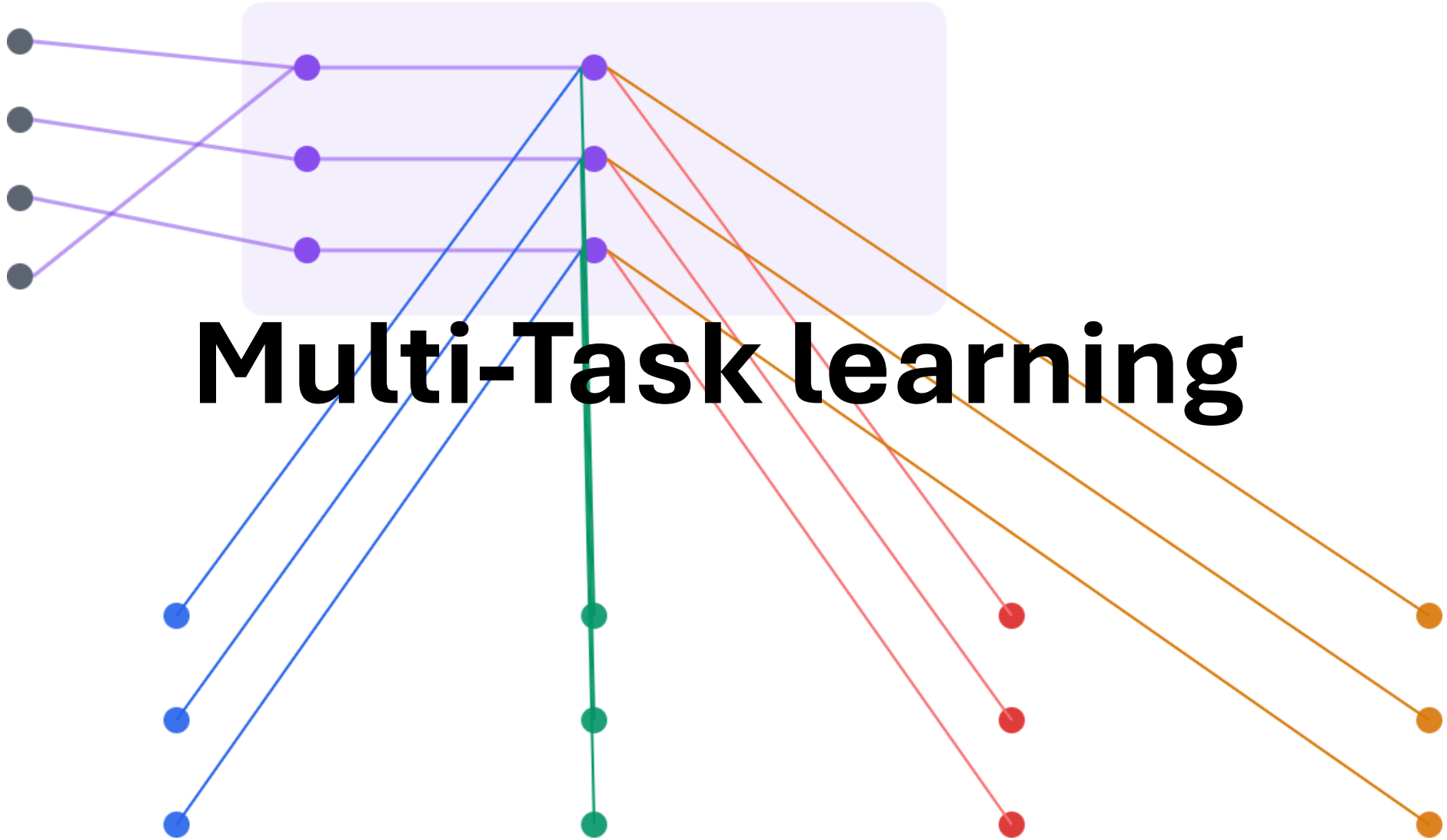
Required Traces: Protected



MLP



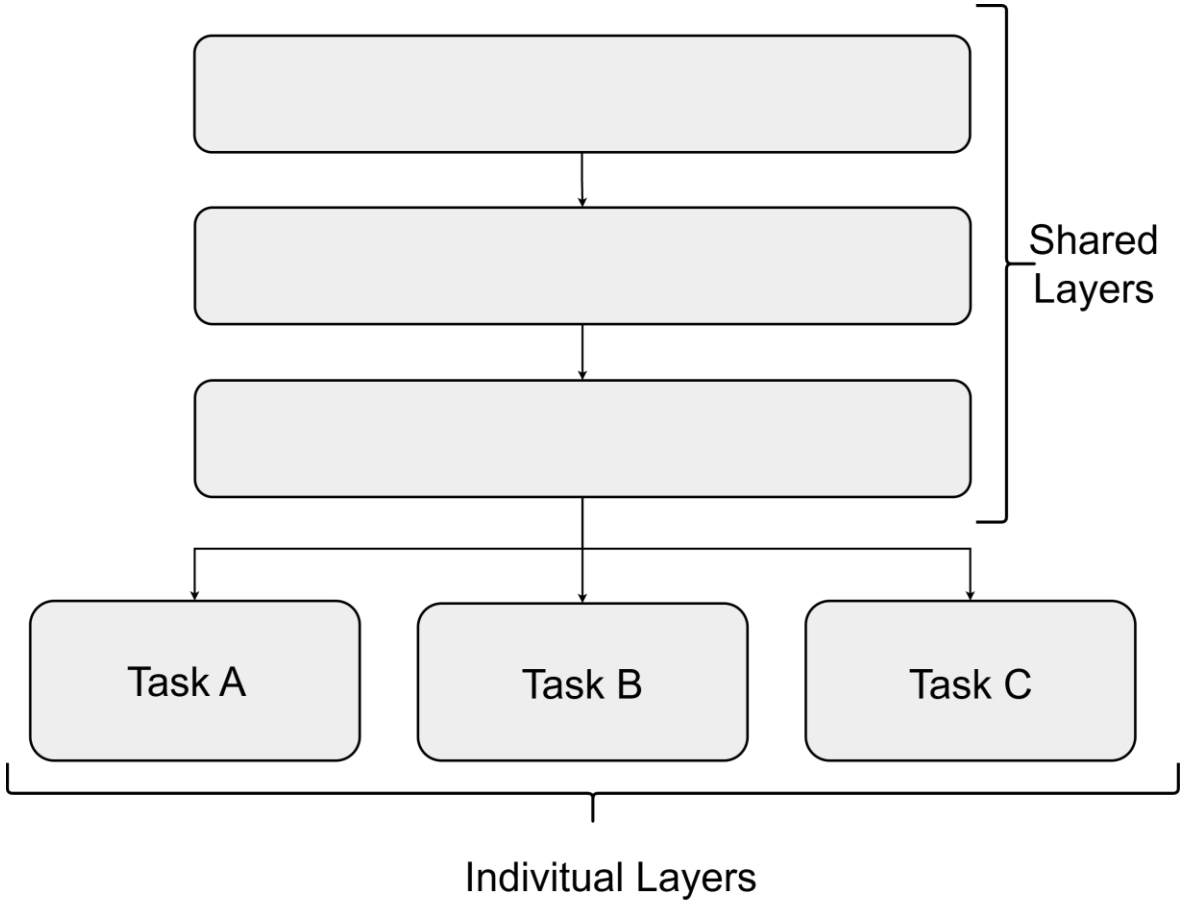
CNN



Multi-Task learning

Multi-Task Learning

- Multi-Task Learning (MTL)
- The underlying assumption:
  - Related tasks contain complementary information
  - This information can be exploited to improve generalization on individual tasks
- MTL reduces the need for redundant parameter learning and enhances efficiency

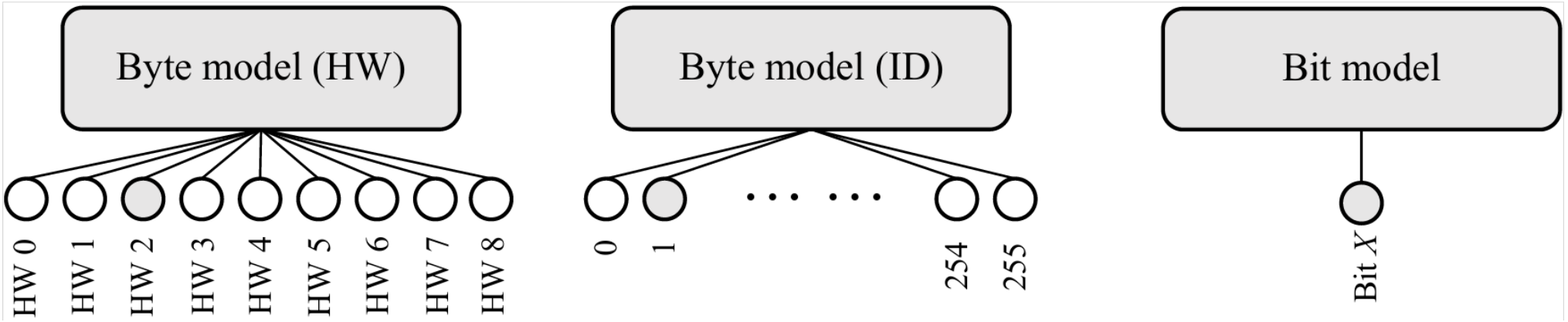


Multi-Task Learning for SCA

- Multiple byte at a time
- The challenge of finding an effective leakage model
  - Stochastic method
  - MTL can implement stochastic method for deep learning
  - It also offers two advantages over stochastic method
  - It can also learn non-linear bit leakage
  - It is possible to learn and attack the whole 16 bytes of a key at once

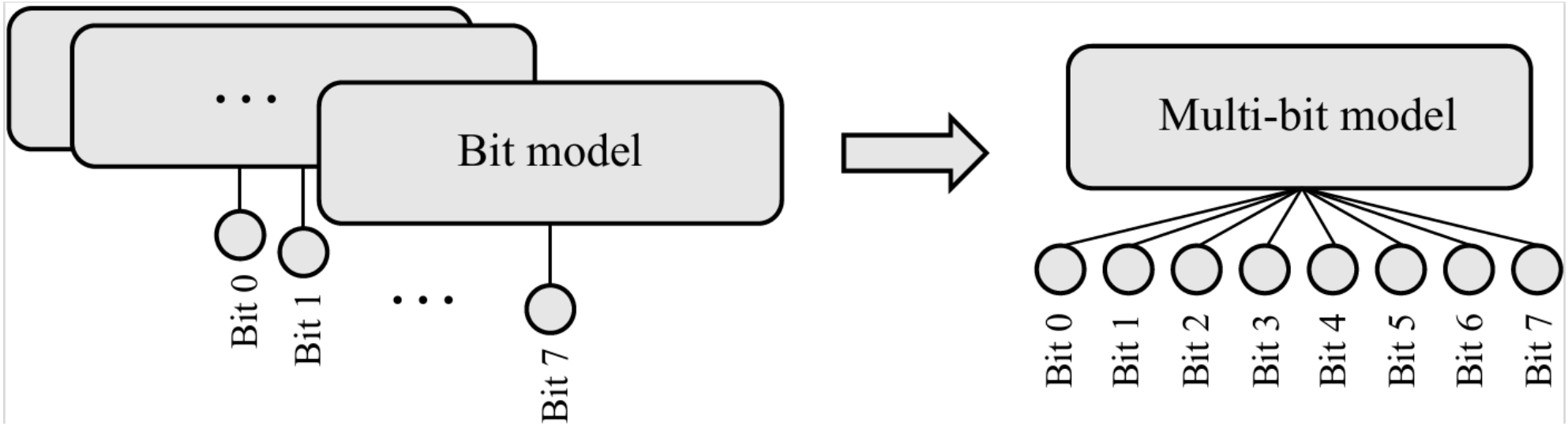
Conventional Pre-determined Leakage Models

Target: One-byte of the intermediate value

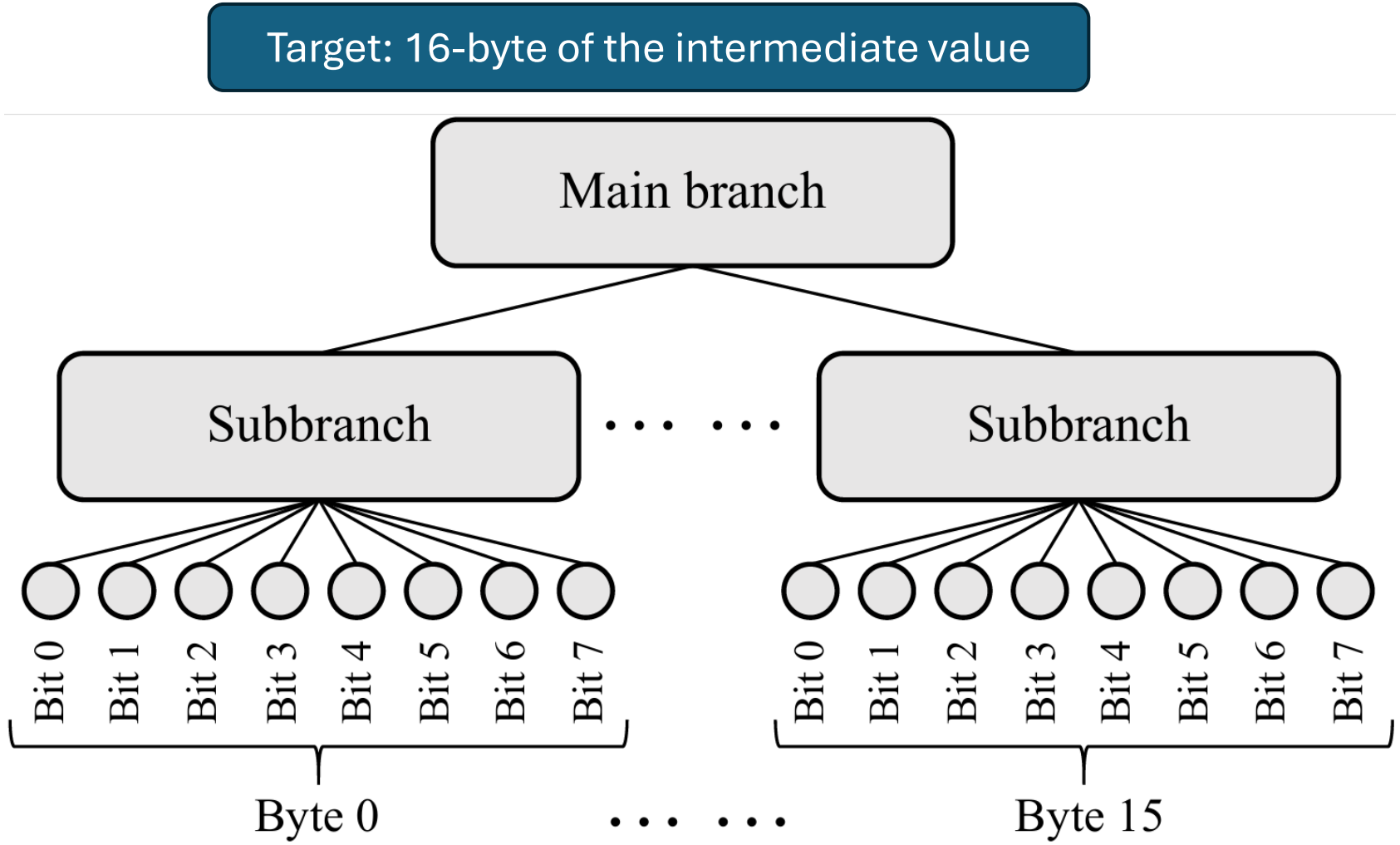


Multi-bits Flexible Leakage Model

Target: One-byte of the intermediate value



Multi-bits Multi-bytes Flexible Leakage Model



GE of each subkey for the ASCAD-F dataset

	k_0	k_1	k_2	k_3	k_4	k_5	k_6	k_7	k_8	k_9	k_{10}	k_{11}	k_{12}	k_{13}	k_{14}	k_{15}
MMB	1	4	7	20	9	4	22	10	135	15	76	18	219	123	9	69
MB	1	4	5	19	9	7	23	17	185	6	37	35	59	67	9	51
HW	4	4	494	282	419	372	535	590	x	628	1 402	792	x	x	202	x
ID	2	2	x	290	66	74	2 415	111	x	126	1 272	x	x	x	23	x
LSB	17	17	1 552	43	63	81	208	57	2 683	49	138	x	x	1 084	44	61
MSB	27	33	x	1 351	1 079	983	984	866	x	623	x	517	x	x	3 544	x

GE of each subkey for the ASCAD-R dataset

	k_0	k_1	k_2	k_3	k_4	k_5	k_6	k_7	k_8	k_9	k_{10}	k_{11}	k_{12}	k_{13}	k_{14}	k_{15}
MMB	3	5	23	76	28	9	15	35	94	14	74	15	177	110	13	48
MB	2	5	23	89	23	10	17	42	80	27	71	16	145	65	15	26
HW	5	6	1 523	1 088	357	439	1 384	1 160	3 482	808	1 081	824	x	x	415	x
ID	3	2	x	x	309	59	x	x	x	x	x	x	x	x	66	x
LSB	16	11	288	118	43	55	192	98	520	39	89	116	x	391	24	54
MSB	39	60	x	x	4 128	x	1 104	x	x	1 698	x	296	x	x	1 023	x

[No Title]

GE of each subkey for the CHES-CTF dataset

	k_0	k_1	k_2	k_3	k_4	k_5	k_6	k_7	k_8	k_9	k_{10}	k_{11}	k_{12}	k_{13}	k_{14}	k_{15}
MMB	89	96	58	100	227	76	48	43	114	55	50	46	3 211	50	59	35
MB	72	57	43	53	78	51	55	56	53	73	40	54	582	40	53	26
HW	2 058	x	803	1 157	x	1 369	1 715	1 121	x	3 013	1 330	4 558	x	x	1 535	1 473
ID	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x
LSB	509	812	140	395	387	501	576	228	266	482	439	267	x	533	581	371
MSB	607	486	1 360	x	2 040	3 164	758	x	x	813	2 257	327	x	x	499	x

GE of each subkey for the eShard dataset

	k_0	k_1	k_2	k_3	k_4	k_5	k_6	k_7	k_8	k_9	k_{10}	k_{11}	k_{12}	k_{13}	k_{14}	k_{15}
MMB	119	504	858	248	1103	691	488	949	2 162	340	106	953	1 503	541	417	470
MB	1 173	942	908	299	x	1 706	1 894	1 085	x	876	2 420	1 459	x	1 269	1 880	1 304
HW	1 690	2 626	1 279	922	x	976	973	783	x	1 742	1 370	1 051	x	1 208	1 557	1 176
ID	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x
LSB	x	x	4 592	2 321	x	x	x	4 640	x	[No Title]	x	x	x	x	x	x
MSB	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x

Now You: Hands-on Exercise

- Please reach the material you need in this link:

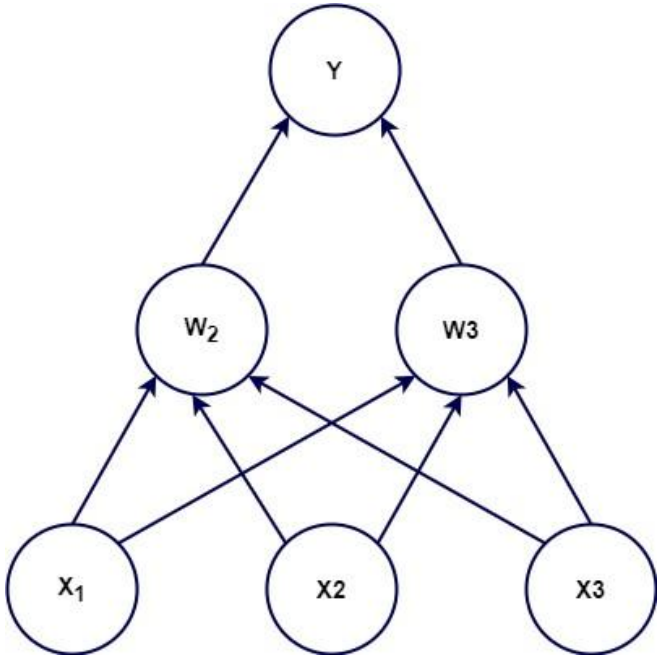
https://github.com/Azi-Lizzy/ii-dlsca

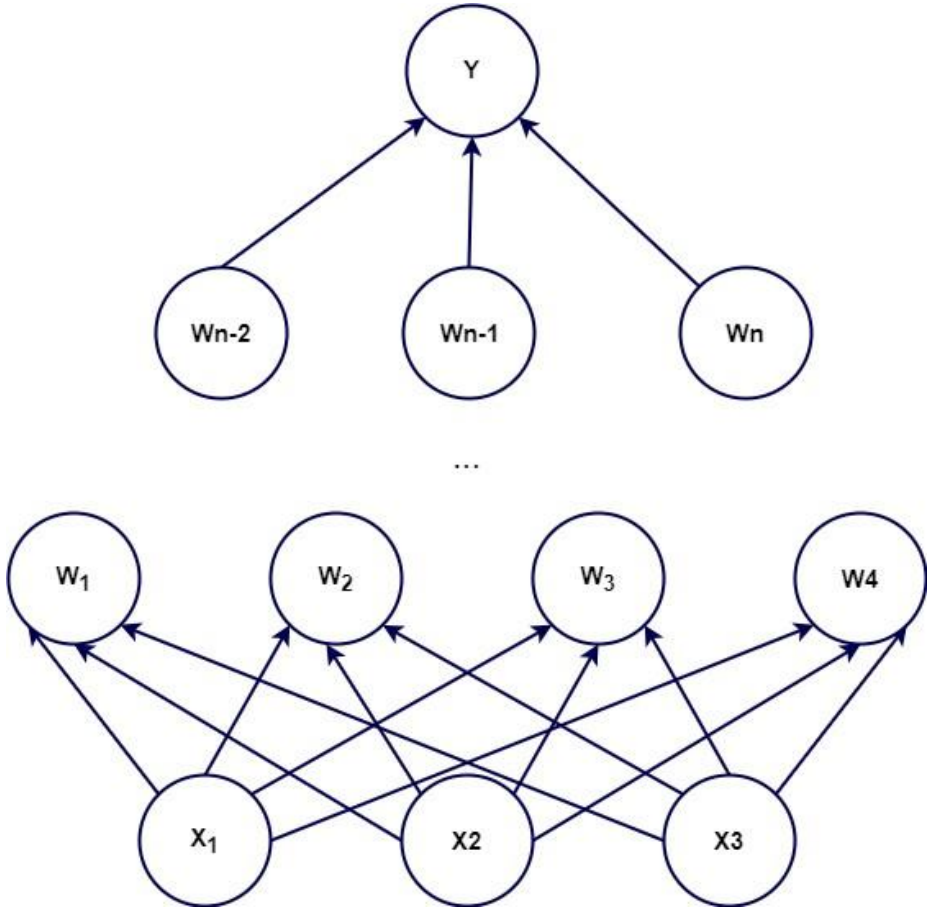
- When opening each .ipynb file you see 

- Open the notebook in google Colab and follow the **TODOs**.


regularization

Deep Neural Networks and Learning Process





Mathematical Perspective of Learning

- Objective Function (Loss Function):

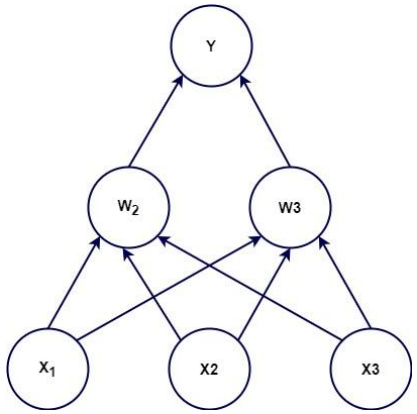
$$\mathrm{E}(W; X, y)$$

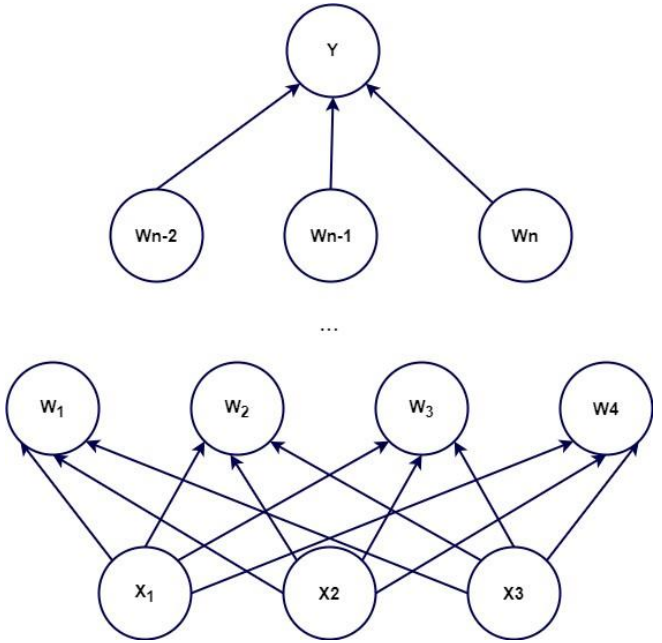
- Example:

$$MSE = \frac{1}{n}\sum_{k=1}^{n}(y_k - \widehat{y_k})^2$$

- Representation Function:

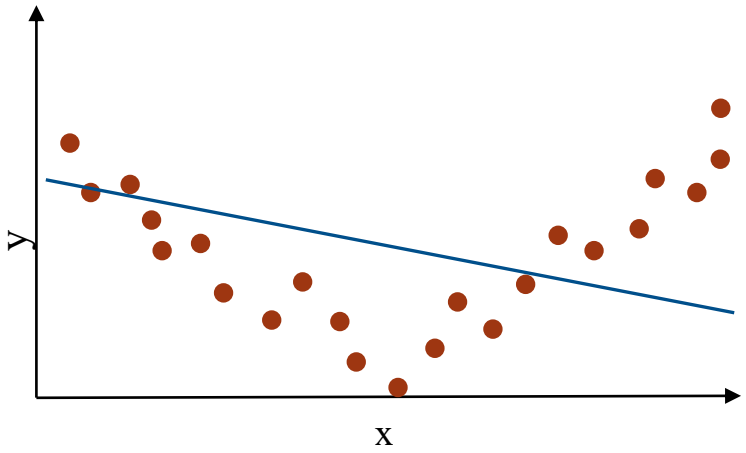
$$\hat{y} = f_W(X)$$





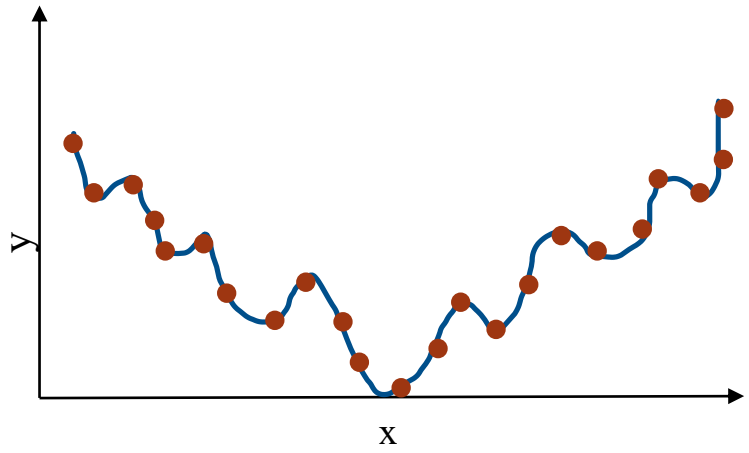
Data and DL-Model Complexity Match

Underfit



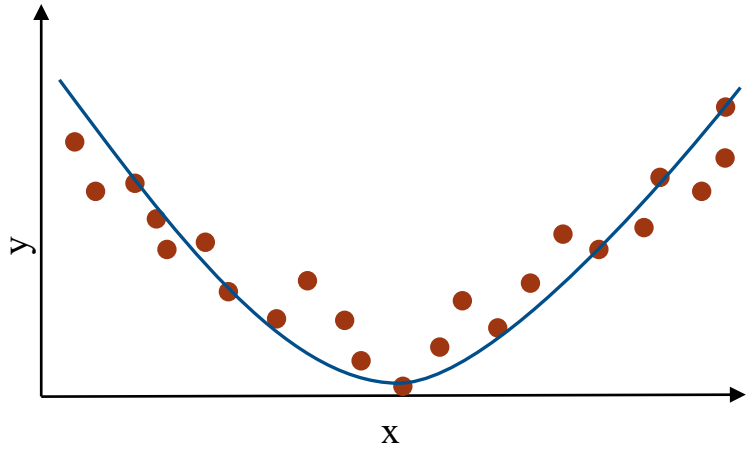
$\hat{y} = w_1 x + w_0$

Overfit



$\hat{y} = w_{30} x^{30} + w_9 x^9 + w_1 x + w_0$

well-generalized



$\hat{y} = w_2 x^2 + w_1 x + w_0$

Regularization Techniques

- Regularization techniques definition:
    - Any modification we make to a learning algorithm tending to reduce test error but not the training error:
- Examples:
    - L_1 regularization
    - L_2 regularization
    - Dropout
    - Early stopping
    - Data augmentation
    - Multi-task learning
    - Ensemble methods

L_1 and L_2 Regularizers

- They both add penalty term to the objective function

$$MSE = \frac{1}{n}\sum_{k=1}^{n}(y_k - \widehat{y_k})^2 + \frac{1}{2}\lambda \sum_{ij} w_{ij}^2$$

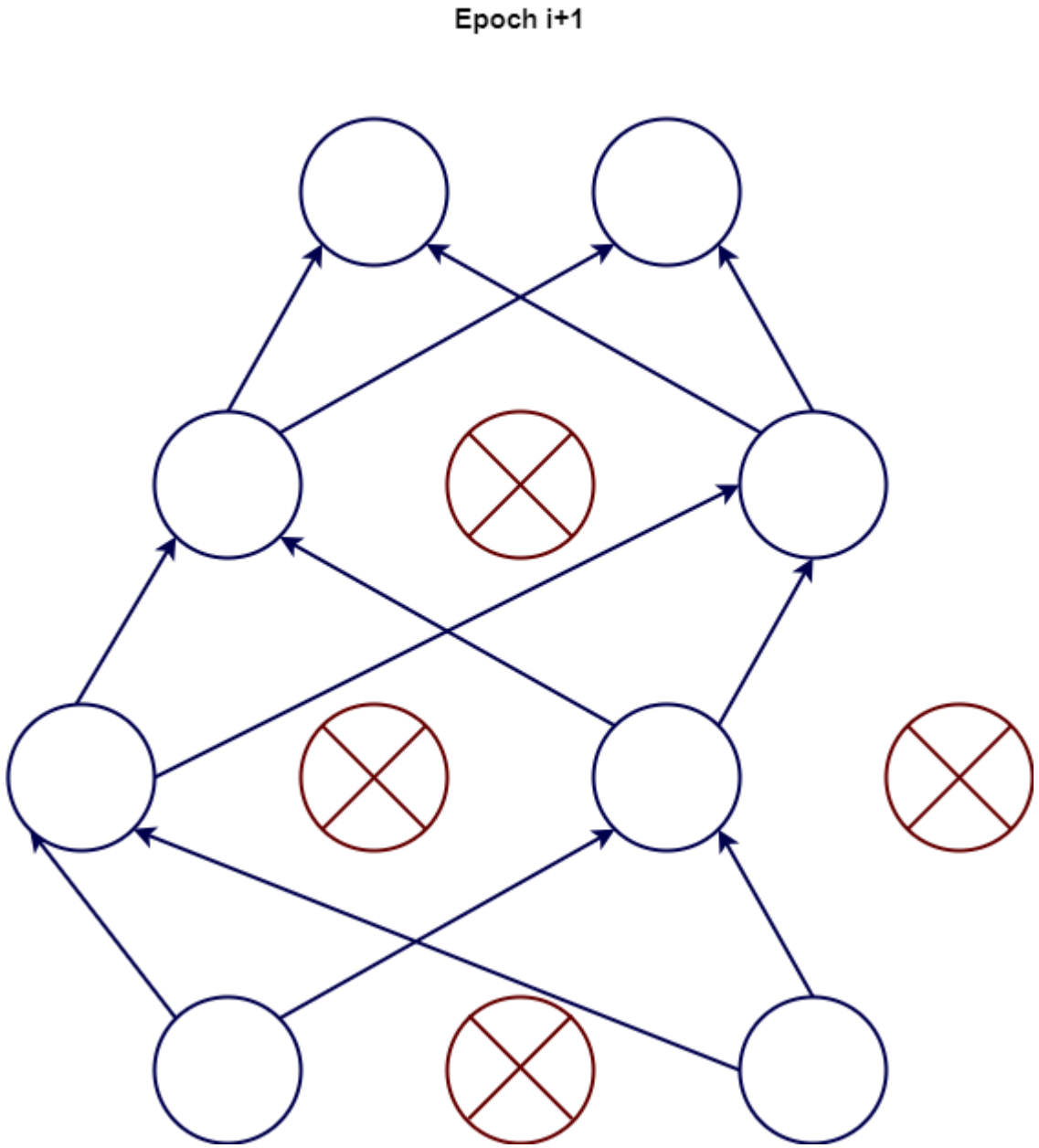
- L_1 zero outs the irrelevant features and is known as a built-in feature selection.

$$MSE = \frac{1}{n}\sum_{k=1}^{n}(y_k - \widehat{y_k})^2 + \frac{1}{2}\lambda \sum_{ij} |W_{ij}|$$

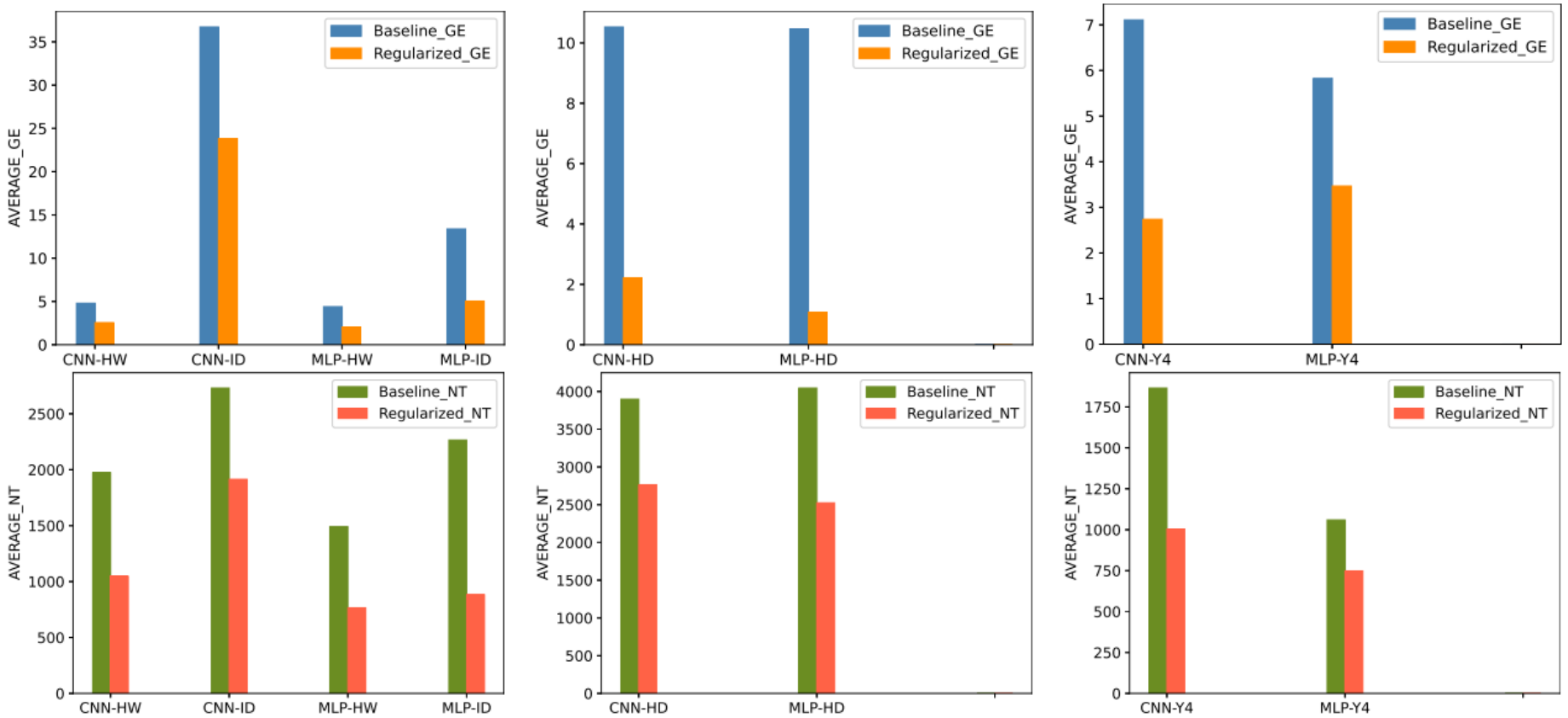
- λ is a parameter governing how strongly large weights are penalized

Dropout

- Temporarily remove random neural unit during training
- Dropout rate q
- It is like training multiple smaller networks and averaging over the weights of these networks for testing

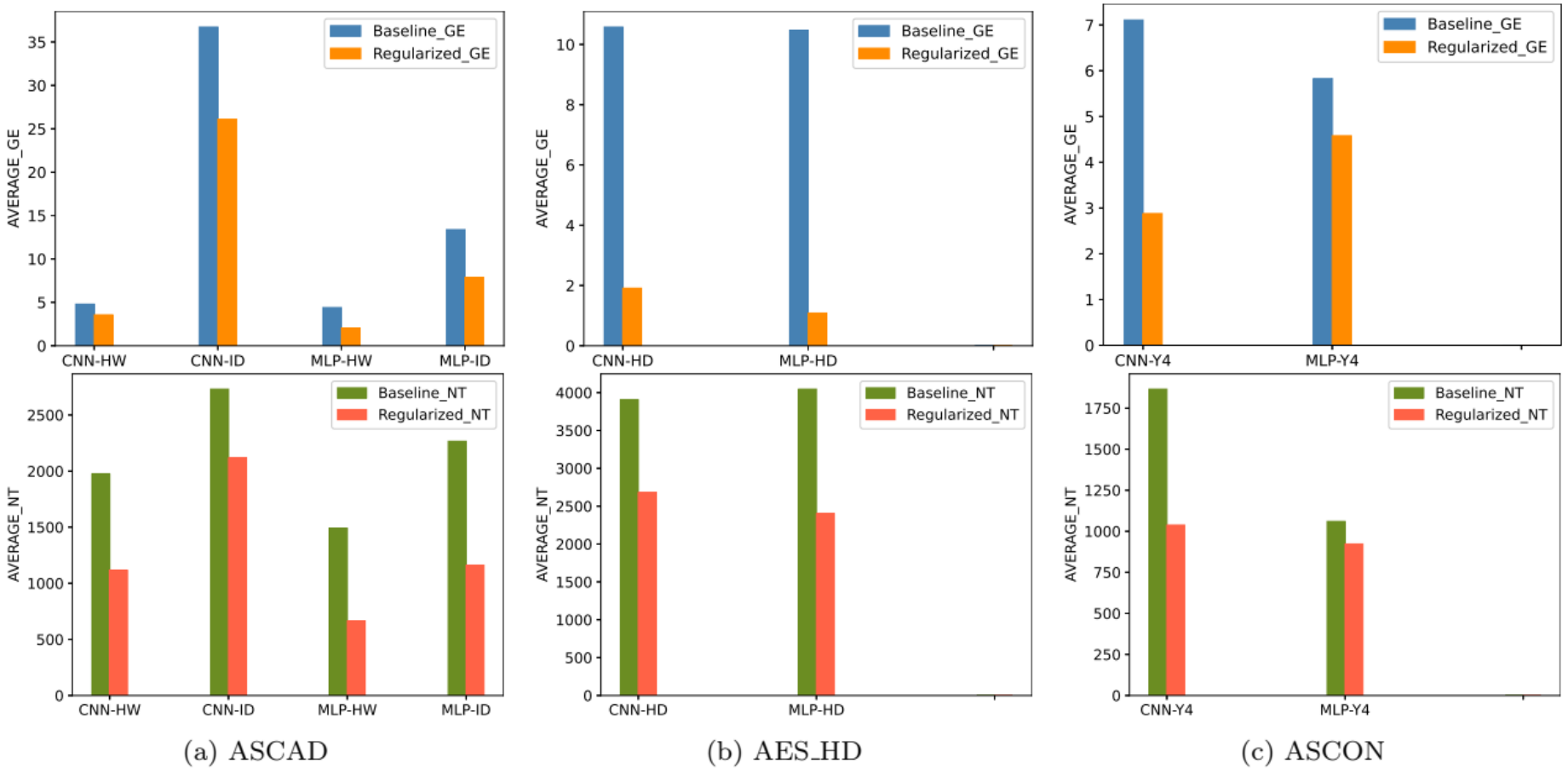


Experimental Results- L_2 Regularization



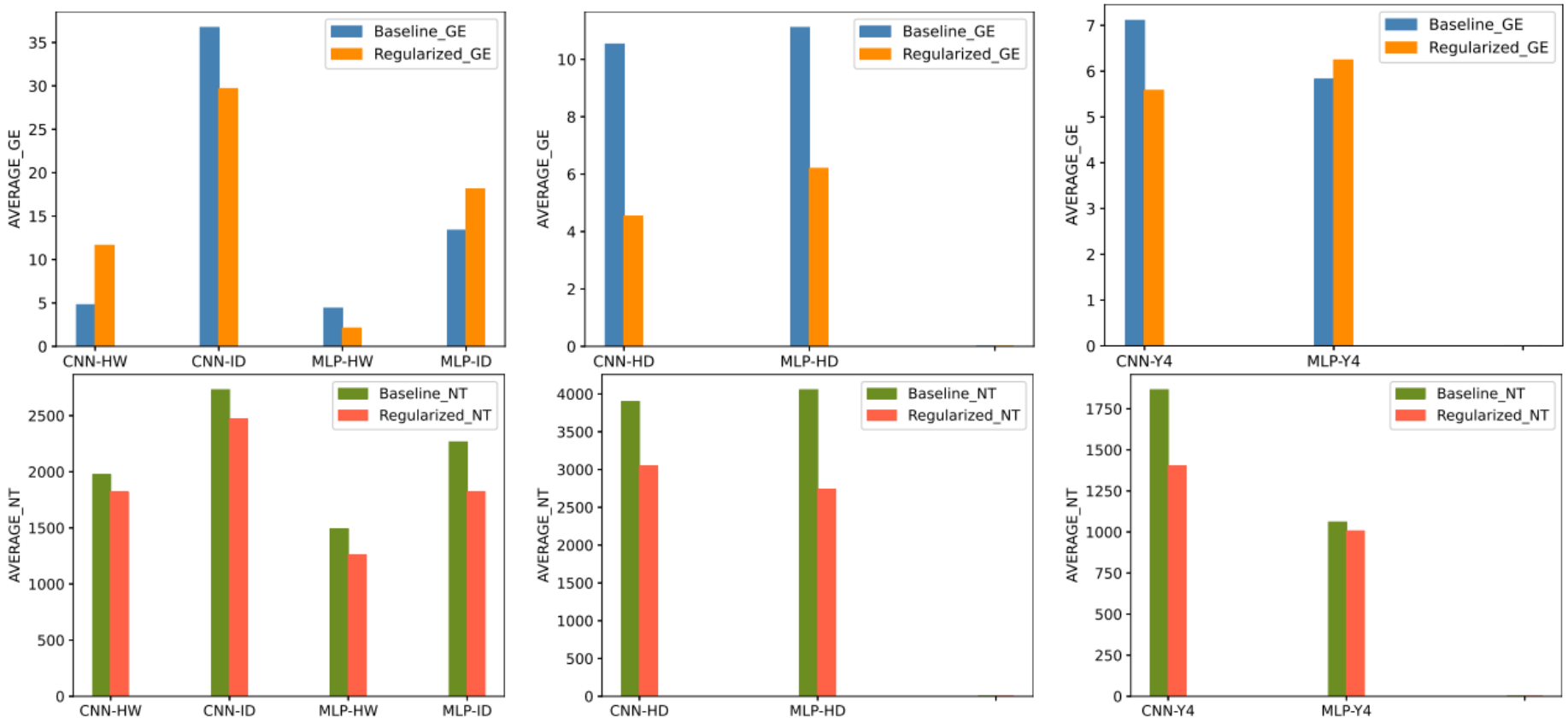
(a) ASCAD (b) AES_HD (c) ASCON

Experimental Results- L_1 Regularization



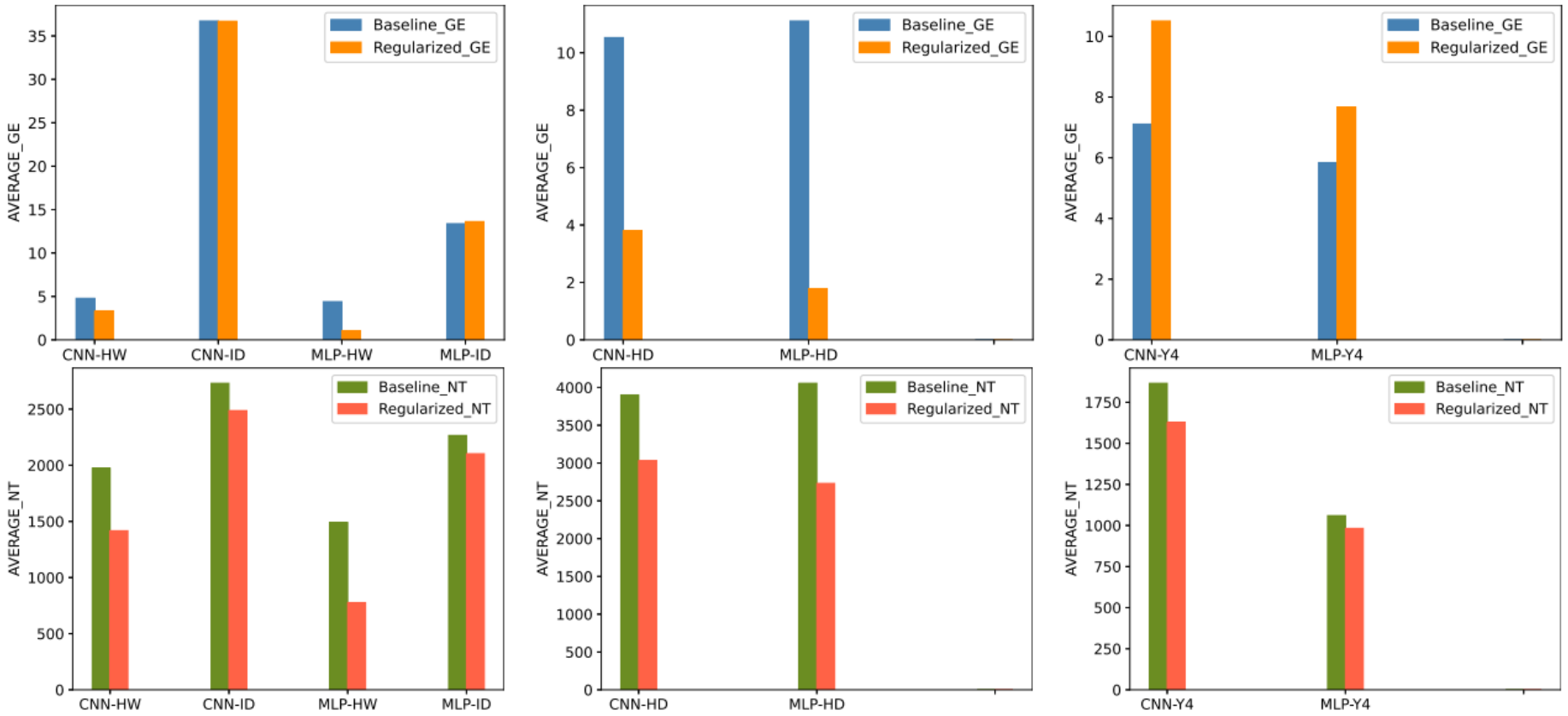
(a) ASCAD (b) AES_HD (c) ASCON

Experimental Results- Dropout



(a) ASCAD (b) AES_HD (c) ASCON

Experimental Results- Early Stopping



(a) ASCAD (b) AES_HD (c) ASCON

Deterioration Rate

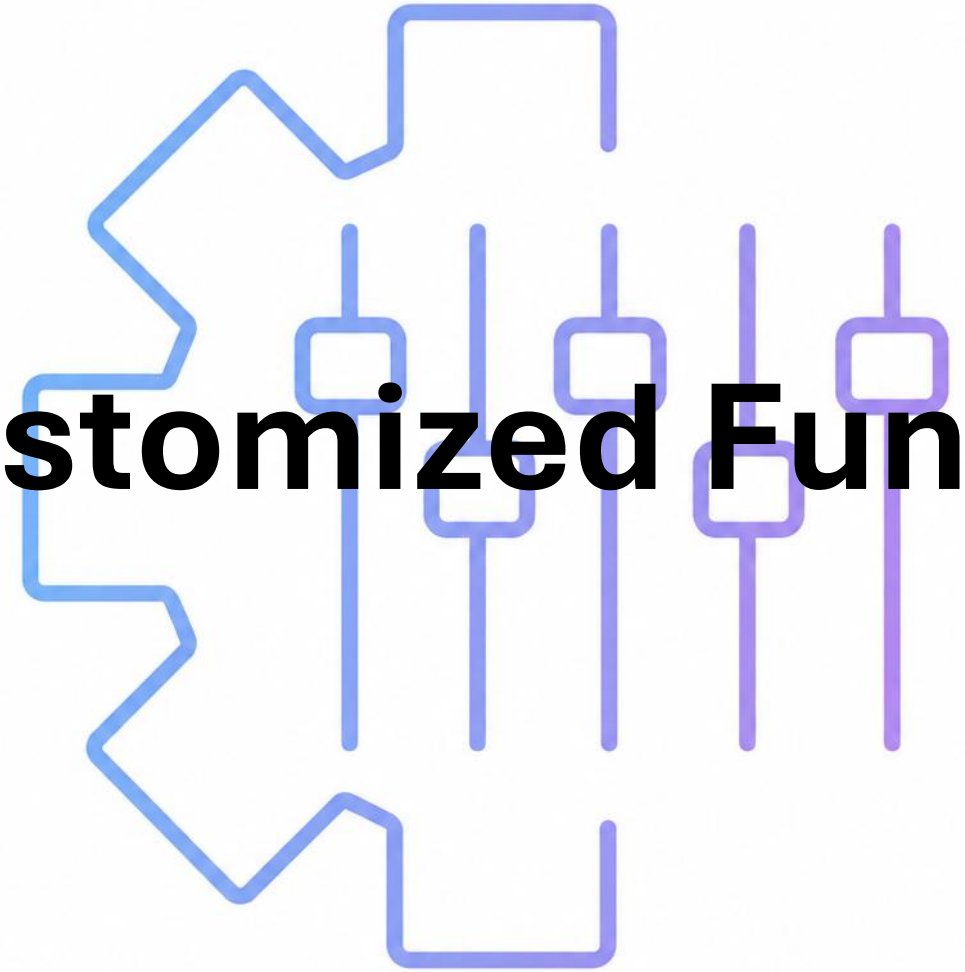
- Definition: the percentage of models that deteriorate after using a regularization technique
- Dropout degrades many regularized models
- Combination including CNN degrade more often

	ASCAD				AES_HD		Unprotected_ASCON	
	CNN-HW	CNN-ID	MLP-HW	MLP-ID	CNN-HD	MLP-HD	CNN-Y4	MLP-Y4
L_1	1.5%	4.5%	0%	6.5%	3%	0.5%	0.5%	9%
L_2	0%	2%	0%	1.5%	2%	0.5%	2%	1.5%
Dropout	29.5%	19%	7.5%	23.5%	22.5%	10.5%	10%	5.5%
Early stopping	8.5%	18%	0.5%	19%	14%	4%	9.5%	2%

Influence on Profiling Time

- Early stopping can reduce profiling time
- The other techniques increase the profiling time

	ASCAD				AES_HD		Unprotected_ASCON	
	CNN-HW	CNN-ID	MLP-HW	MLP-ID	CNN-HD	MLP-HD	CNN-Y4	MLP-Y4
Baseline models	933	986	651	663	825	465	16	12
L_1	3739	4620	1854	2603	6027	4832	23	18
L_2	3934	3958	1396	2097	6881	4194	28	17
Dropout	2420	3299	1209	1626	3759	1967	25	16
Early stopping	632	898	165	579	432	256	14	9



Customized Functions

Difference Between DL and DL-SCA

- Performance metrics are different
- Accuracy and loss
- Compared to guessing entropy and required number of attack traces
- Added noise by design

We Need Customization

- The difference in DL-SCA t nature compared to regular DL leads to customization requirement
- For example:
  - Customized loss function
    - Ranking loss
    - Focus
  - Or specific activation function
    - Jump-Relu
  - Or Specific optimization

Now You: Hands-on Exercise

- Please reach the material you need in this link:

https://github.com/Azi-Lizzy/ii-dlsca

- When opening each .ipynb file you see 

- 03_Regularizations.ipynb
- Open the notebook in google Colab and follow the **TODOs**.

Wrap up

